

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Sistemas Operativos



Grupo 1

Ing. Angel Isidro Mercado

Proyecto: Instalación de sistema operativo Linux

320304435 - Cabrera Rojas Oscar

Ciudad Universitaria, CDMX a 06 de septiembre del 2026

Índice

1. Fundamentos de almacenamiento y particionamiento de discos **1**
1.1. Geometría de almacenamiento secundario y direccionamiento de bloques . . 1
1.2. Esquemas de tablas de particiones: MBR frente a GPT 3
1.2.1. Arquitectura y limitaciones del registro de arranque maestro (MBR) . 3
1.2.2. Arquitectura y ventajas de la tabla de particiones GUID (GPT) 4
1.3. Tipologías de particiones y puntos de montaje en Linux 6
1.3.1. Partición del sistema EFI (ESP) 7
1.3.2. Partición o directorio de arranque (`/boot`) 7
1.3.3. Partición raíz (`/`) 8
1.3.4. Partición de datos de usuario (`/home`) 8
1.3.5. Espacio y mecanismos de intercambio (*swap*) 9

2. Arquitectura de arranque y coexistencia de sistemas operativos **10**
2.1. Interfaces de firmware: BIOS heredado y UEFI 10
2.2. Mecanismos de arranque múltiple: Windows Boot Manager y GNU GRUB . . 12
2.3. Factores críticos de coexistencia: Secure Boot, Fast Startup e hibernación NTFS 14
2.3.1. El protocolo Secure Boot y la cadena de confianza criptográfica . . . 14
2.3.2. Riesgo de corrupción por Windows Fast Startup e hibernación de NTFS 15
2.3.3. Sincronización del reloj de hardware en tiempo real (RTC) 15

3. Requerimientos de hardware y preparación del entorno **16**
3.1. Especificaciones técnicas mínimas y arquitectura de soporte en Arch Linux . 16
3.2. Diagnóstico, verificación de compatibilidad e integridad criptográfica de la ISO 18
3.3. Estrategias de respaldo y mitigación de riesgos de pérdida de datos 20
3.3.1. Capa 1: Respaldo del almacén BCD y de la partición ESP 21
3.3.2. Capa 2: Respaldo de metadatos de particionamiento con sgdisk . . . 21
3.3.3. Capa 3: Integridad de NTFS y reducción del volumen desde Windows 21

4. Herramientas de particionamiento y gestión de almacenamiento **22**
4.1. Herramientas nativas de Windows: Administrador de discos y DiskPart 22
4.1.1. El Administrador de discos gráfico (diskmgmt.msc) 23
4.1.2. La utilidad de línea de comandos DiskPart 23
4.2. Utilidades de línea de comandos en Linux: fdisk, gdisk y parted 24
4.2.1. util-linux fdisk 24
4.2.2. GPT fdisk (gdisk) 24
4.2.3. GNU Parted 25
4.3. Entornos gráficos y herramientas en vivo: GParted 26
4.3.1. Arquitectura modular de GParted 26
4.3.2. El modelo de transacciones en cola diferida 27
4.3.3. Mecánica física de reubicación y redimensionamiento de volúmenes . 27

5. Protocolo técnico de instalación y configuración paso a paso **28**
5.1. Generación del medio de arranque y verificación del modo de inicio UEFI . . 28
5.2. Particionamiento manual y formateo de sistemas de archivos 29
5.3. Jerarquía de montaje en /mnt y despliegue base (pacstrap) 30
5.4. Configuración del entorno en chroot 31

5.4.1. Sincronización horaria y localización geográfica 31
5.4.2. Identidad de red, seguridad y servicios del sistema 32
5.4.3. Generación de la imagen de disco en memoria inicial (initramfs) . . . 32
5.5. Instalación de GNU GRUB con soporte para Windows Boot Manager (os-prober) 32

6. Postinstalación, configuración del sistema y resolución de fallos 33
6.1. Gestión de paquetes con Pacman y modularidad en la configuración del entorno 33
6.2. Ajuste del cargador de arranque y sincronización de reloj de hardware 34
6.3. Diagnóstico y resolución de incidencias habituales en arranque dual 35
6.3.1. Sobrescritura del orden de arranque por actualizaciones de Windows 35
6.3.2. Reactivación accidental de Fast Startup y volumen NTFS sucio 36

7. Conclusiones personales 36

Referencias 37

1. Fundamentos de almacenamiento y particionamiento de discos

La correcta instalación, configuración y coexistencia armónica de múltiples sistemas operativos en una misma estación de cómputo descansa sobre la comprensión rigurosa de las capas inferiores del subsistema de almacenamiento secundario. Lejos de constituir un simple receptáculo homogéneo de bytes, una unidad de almacenamiento masivo —ya sea electromecánica o basada en memoria de estado sólido— requiere una jerarquía estricta de estructuración que abarca desde la traducción física de señales y bloques hasta la definición de tablas de particiones y la organización lógica de árboles de archivos en el sistema operativo (Stallings, 2018, §11.5, pp. 521–523).

1.1. Geometría de almacenamiento secundario y direccionamiento de bloques

Históricamente, los discos duros magnéticos estructuraban su capacidad de almacenamiento a partir de una geometría física basada en platos circulares cubiertos de material magnetizable. Como describen Stallings (2018, §11.5, pp. 521–523), cada plato dispone de una o dos superficies de grabación leídas y escritas por cabezales electromagnéticos suspendidos microscópicamente sobre ellas. La superficie se subdivide concéntricamente en pistas (*tracks*), y el conjunto vertical de pistas situadas en el mismo radio a través de todos los platos conforma un cilindro (*cylinder*). Cada pista se segmenta a su vez en arcos denominados sectores (*sectors*), los cuales constituyen la unidad indivisible más pequeña de transferencia atómica en el medio físico.

Para que el controlador del disco y el sistema de entrada/salida (E/S) puedan operar sobre el soporte, el disco debe someterse a un formateo físico o de bajo nivel (Silberschatz et al., 2006, §12.5.1, pp. 418–419). Durante este procedimiento, el fabricante delimita los sectores inscribiendo en cada uno una cabecera con información de sincronización y número de sector, un área de datos de carga útil y una cola con códigos de corrección de errores (*error-correcting code*, ECC). El algoritmo de ECC recalcula periódicamente los polinomios verificadores tras cada lectura, lo que permite detectar corrupciones de bits y recuperar errores transitorios antes de que se propaguen a las capas superiores del sistema operativo. Aunque durante décadas el tamaño estándar de la carga útil del sector físico fue fijado en 512 bytes, las unidades modernas adoptan el estándar de formato avanzado (*Advanced Format*, AF), incrementando el tamaño físico del sector a 4096 bytes (4 KiB). Esta ampliación reduce drásticamente el espacio muerto consumido por cabeceras y colas repetitivas, y eleva la eficiencia de los algoritmos de detección y corrección de errores. Para preservar la compatibilidad con plataformas heredadas, los controladores de almacenamiento ofrecen emulación de 512 bytes (formato 512e), mientras que los sistemas nativos operan directamente con direccionamiento de 4 KiB nativo (4Kn).

En las primeras arquitecturas de computadoras personales, el acceso a los sectores demandaba la especificación directa de sus coordenadas espaciales mediante la terna de cilindro, cabezal y sector (CHS, *Cylinder-Head-Sector*). En este esquema, el sistema operativo o el firmware del sistema básico de entrada/salida (*Basic Input/Output System*, BIOS) instruía al controlador mediante llamadas a la interrupción INT 13h, especificando con exactitud tridimensional qué cabezal activar y sobre cuál cilindro y sector posicionarse (Carretero Pérez et al., 2001, §7.4, pp. 377–378). No obstante, el esquema CHS arrastraba limitacio-

nes intrínsecas impuestas por la combinación de las restricciones de los registros de la controladora IDE/ATA y las estructuras de parámetros de disco de la BIOS tradicional. Los registros originales reservaban:

- 10 bits para el número de cilindro, limitando el recorrido a $2^{10} = 1024$ cilindros (numerados del 0 al 1023).
- 4 bits en la interfaz ATA (ampliados a 8 bits en la BIOS mediante traducción) para el cabezal, permitiendo hasta $2^4 = 16$ cabezales físicos (o 256 en llamadas de BIOS).
- 6 bits para el sector por pista, con la particularidad de que los sectores se numeran a partir de 1, admitiendo un máximo de $2^6 - 1 = 63$ sectores.

El producto combinatorio de estas restricciones fijó una severa barrera histórica de almacenamiento:

$$\text{Capacidad}_{\text{máx CHS-BIOS}} = 1024 \times 16 \times 63 \times 512 \text{ bytes} \approx 528.48 \text{ MB} \quad (504 \text{ MiB}) \tag{1}$$

Incluso tras implementar esquemas de traducción asistida de geometría mediante software y controladores mejorados (Enhanced BIOS), el límite superior de CHS no pudo extenderse más allá de los 8.4 GB ($1024 \times 256 \times 63 \times 512$ bytes).

Para superar definitivamente estas restricciones mecánicas y desacoplar el diseño lógico del software de las contingencias de fabricación de los discos, se estandarizó el direccionamiento de bloques lógicos (LBA, *Logical Block Addressing*). Bajo LBA, el disco completo es abstraído ante el sistema operativo como una matriz lineal unidimensional de bloques homogéneos, numerados secuencialmente desde el índice 0 hasta $N - 1$, donde N representa el número total de sectores disponibles en la unidad (Silberschatz et al., 2006, §12.1, p. 409). El sector 0 corresponde lógicamente al primer sector de la primera pista del primer cilindro exterior, avanzando de manera continua sin que el sistema operativo deba calcular pasos de pista o conmutaciones de cabezal. La conversión interna entre el índice LBA solicitado y las coordenadas físicas reales queda delegada de forma transparente al microcontrolador integrado en la unidad de almacenamiento.

Tabla 1: Evolución histórica de los esquemas de direccionamiento de almacenamiento.

Esquema	**Bits**	**Sectores**	**Sector**	**Capacidad máx.**	**Contexto tecnológico**
CHS estándar	24	1 032 192	512 B	504 MiB	BIOS heredado e interfaces IDE primigenias.
CHS asistido	28	16 515 072	512 B	8.41 GB	Traducción de geometría (EBIOS / ATA-1).
LBA28	28	2^{28}	512 B	137.44 GB (128 GiB)	Norma ATA-1 a ATA-5; barrera de los 137 GB.
LBA48	48	2^{48}	512 B	144.11 PB (128 PiB)	Norma ATA-6 / SATA / SAS; estándar extendido.
LBA48 (4Kn)	48	2^{48}	4096 B	1.15 EB (1 ZiB)	Formato avanzado nativo para unidades masivas.

La transición a LBA no solo resolvió las restricciones de escala en discos mecánicos, sino que sentó las bases para la integración de las unidades de estado sólido (*Solid-State Drive*, SSD) y los dispositivos basados en el protocolo de memoria no volátil exprés (*Non-Volatile Memory Express*, NVMe). En los medios de estado sólido, no existen superficies giratorias ni cabezales mecánicos; el almacenamiento descansa en celdas de memoria

flash NAND organizadas en matrices de páginas (típicamente de 4 KiB a 16 KiB) y bloques de borrado (de 2 MiB a 8 MiB). El controlador del SSD incorpora una capa de traducción de memoria flash (*Flash Translation Layer*, FTL), la cual expone hacia el sistema operativo la interfaz clásica de bloques LBA de 512 o 4096 bytes, mientras administra dinámicamente el desgaste de celdas (*wear leveling*), la recolección de basura y la reubicación de bloques lógicos.

Este desacoplamiento impone un requisito crítico de diseño en la fase de particionamiento: la alineación de sectores. Si una partición inicia en un sector LBA no divisible entre el tamaño del bloque físico de la memoria flash o del sector 4Kn, cada operación de lectura o escritura de un bloque del sistema de archivos se desalinea, forzando al controlador a acceder a dos bloques físicos contiguos para completar una única operación lógica. Este fenómeno, denominado amplificación de escritura, degrada sustancialmente el rendimiento de entrada/salida y reduce la vida útil de los circuitos integrados de la unidad. Por esta razón, las herramientas contemporáneas de particionamiento estandarizan la alineación forzada de cualquier partición en fronteras de 1 MiB (exactamente en el LBA 2048 para sectores de 512 bytes, puesto que 2048×512 bytes $= 1\,048\,576$ bytes $= 1$ MiB), garantizando divisibilidad exacta con cualquier tamaño común de página física o bloque de borrado.

1.2. Esquemas de tablas de particiones: MBR frente a GPT

El particionamiento es el proceso de segmentación de la matriz global de bloques LBA de un medio de almacenamiento en subconjuntos contiguos de sectores lógicamente aislados, a los cuales el sistema operativo trata como si fuesen unidades físicas independientes (Silberschatz et al., 2006, §12.5.1, pp. 418–419). Cada partición delimita un espacio continuo delimitado por un sector de inicio y un sector de término, dentro del cual se puede implantar un sistema de archivos independiente, un área de memoria virtual o una estructura de arranque (Carretero Pérez et al., 2001, §7.4, pp. 377–378). La definición, orden y atributos de estas regiones se almacenan en una estructura denominada tabla de particiones. En la computación contemporánea coexisten dos esquemas fundamentales: el registro de arranque maestro (MBR, *Master Boot Record*) y la tabla de particiones GUID (GPT, *GUID Partition Table*).

1.2.1. Arquitectura y limitaciones del registro de arranque maestro (MBR)

Diseñado a inicios de la década de 1980 por IBM y Microsoft para las computadoras personales PC-XT, el esquema MBR ubica su estructura exclusivamente en el primer sector absoluto del disco (LBA 0), abarcando exactamente 512 bytes. La composición interna de este sector se divide en tres campos fundamentales:

1. **Código de arranque primario (446 bytes):** Rutina en código máquina que el firmware BIOS carga en memoria principal (típicamente en la dirección física `0x7C00`) para iniciar la primera etapa del cargador de arranque (*bootloader*).
2. **Tabla de particiones primarias (64 bytes):** Estructura formada por cuatro entradas contiguas de 16 bytes cada una. Cada entrada describe el estado de activación (booteable o no), el cilindro-cabezal-sector de inicio, el tipo de sistema de archivos (código hexadecimal de 1 byte), las coordenadas CHS de fin, el LBA inicial y el número total de sectores que integran la partición.
3. **Firma mágica de arranque (2 bytes):** La secuencia hexadecimal `0x55AA` en los desplazamientos 510 y 511, empleada por el BIOS para certificar que el sector contiene

código de arranque válido.

Debido a que la tabla dedica exactamente 16 bytes a cada registro y solo cuenta con 64 bytes disponibles, el MBR impone la restricción estricta de albergar únicamente hasta cuatro particiones denominadas primarias. Para eludir esta restricción sin modificar la estructura del sector 0, la industria introdujo el concepto de partición extendida. Una partición extendida es un tipo especial de partición primaria (identificada con el código `0x05` o `0x0F`) que actúa como un contenedor secundario. Dentro de la partición extendida se encadenan registros de arranque extendidos (EBR, *Extended Boot Record*) organizados en una lista enlazada simple, donde cada EBR describe una subpartición denominada lógica y apunta al sector donde reside el siguiente EBR de la cadena (Carretero Pérez et al., 2001, §7.4, pp. 377–378).

No obstante, la debilidad más insalvable del esquema MBR radica en el tamaño de los campos destinados a registrar las direcciones de sector. Tanto el campo de sector de inicio como el de conteo de sectores disponen de una longitud de solo 32 bits (4 bytes). Con 32 bits, el valor numérico más alto direccionable es $2^{32} = 4\,294\,967\,296$ sectores. Considerando el tamaño estándar de sector de 512 bytes, la capacidad máxima total direccionable por una tabla MBR es:

$$\text{Capacidad}_{\text{máx MBR}} = 2^{32} \times 512 \text{ bytes} = 2\,199\,023\,255\,552 \text{ bytes} \approx 2.199 \text{ TB} \quad (2.00 \text{ TiB}) \quad (2)$$

Cualquier sector situado más allá del límite de los 2 TiB resulta matemáticamente invisible e inalcanzable para una tabla de particiones MBR. Asimismo, el MBR carece por completo de redundancia: si el sector LBA 0 sufre daño físico o corrupción por escritura errónea, la totalidad de la información sobre las particiones se pierde de manera catastrófica. Tampoco implementa sumas de verificación ni códigos de detección de errores sobre sus propios metadatos, lo que impide al sistema verificar si las entradas han sido alteradas o dañadas.

1.2.2. Arquitectura y ventajas de la tabla de particiones GUID (GPT)

Como parte integrante de la especificación de la interfaz de firmware extensible unificada (UEFI, *Unified Extensible Firmware Interface*), se desarrolló el estándar de tabla de particiones GUID (GPT), diseñado para erradicar las vulnerabilidades del MBR y responder a las exigencias de almacenamiento de la computación contemporánea (Unified Extensible Firmware Interface Forum, 2024).

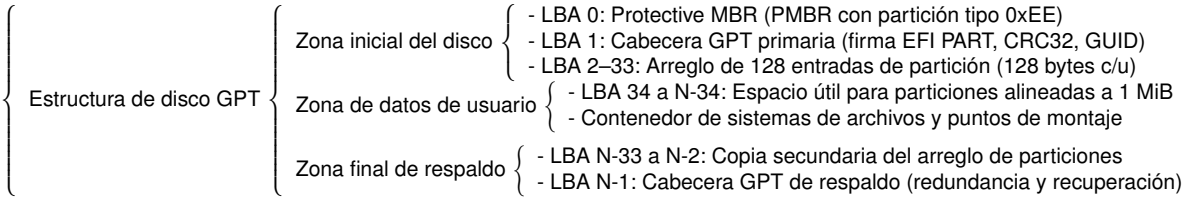


Imagen 1: Disposición estructural de sectores lógicos en un esquema GPT bajo norma UEFI.

A diferencia del MBR, la arquitectura GPT se despliega de forma estructurada a lo largo de los extremos del medio de almacenamiento, reservando bloques específicos para salvaguardar la integridad de los datos (Imagen 1):

- **MBR de protección (LBA 0):** Para garantizar interoperabilidad y protección frente a utilidades de disco heredadas, el sector LBA 0 aloja un MBR de protección (*Protective*

MBR, PMBR). Este sector contiene una tabla de particiones con una única entrada ficticia de tipo `0xEE` que declara ocupar la totalidad de la unidad (o hasta el límite de 2 TiB). Si una utilidad anticuada que desconoce GPT inspecciona el disco, interpreta que este se encuentra totalmente asignado a un volumen desconocido y rehúsa sobreescribir la estructura, previniendo la destrucción accidental de los metadatos (Unified Extensible Firmware Interface Forum, 2024).

- **Cabecera GPT primaria (LBA 1):** Ubicada inmediatamente después del PMBR, define los parámetros operativos de la tabla. Almacena la firma mágica `EFI PART` (en hexadecimal `0x5452415020494645`), la versión de revisión de la especificación, el tamaño de la cabecera (típicamente 92 bytes), un identificador único global (GUID) del disco, el LBA actual de la cabecera (LBA 1), el LBA donde reside la cabecera de respaldo (LBA $N-1$), el rango de bloques utilizables para datos de usuario (`FirstUsableLBA` usualmente fijado en LBA 34, y `LastUsableLBA` en LBA $N-34$), el sector de inicio del arreglo de entradas de partición (LBA 2), el número de entradas reservadas (estandarizado en 128 entradas) y el tamaño de cada entrada (128 bytes). Fundamentalmente, la cabecera incorpora una suma de comprobación de redundancia cíclica de 32 bits (CRC32) de la propia cabecera y otra CRC32 del arreglo completo de particiones. Si ocurre corrupción de bits, el firmware detecta la alteración antes de iniciar cualquier subsistema.
- **Arreglo de entradas de partición primaria (LBA 2 al 33):** Ocupa 32 sectores consecutivos de 512 bytes ($32 \times 512 = 16\,384$ bytes $= 16$ KiB). Aloja 128 registros individuales de partición de 128 bytes cada uno. Cada entrada incluye:
  - GUID de tipo de partición (16 bytes): define el propósito formal del volumen de manera universalmente inequívoca.
  - GUID único de partición (*Unique Partition GUID* o PARTUUID, 16 bytes): identificador aleatorio asignado de manera irrepetible a la partición específica.
  - LBA de inicio y LBA de fin (8 bytes cada uno, enteros de 64 bits): permite referenciar sectores hasta 2^{64}.
  - Atributos de partición (8 bytes): banderas de 64 bits para marcar atributos como volumen de solo lectura, oculto o de arranque del sistema.
  - Nombre de la partición (72 bytes): cadena de hasta 36 caracteres codificada en UTF-16LE, permitiendo etiquetar los volúmenes directamente en la tabla.
- **Estructura de respaldo secundaria (LBA $N-33$ al $N-1$):** En el extremo final de la unidad se duplica íntegramente la estructura: los sectores LBA $N-33$ a $N-2$ albergan la copia de seguridad del arreglo de entradas de partición, y el sector LBA $N-1$ almacena la cabecera GPT de respaldo. Si la cabecera primaria se corrompe o sufre una escritura incompleta por un fallo eléctrico, el firmware UEFI o utilidades como `gdisk` restauran automáticamente el estado consistente empleando la copia secundaria.

Gracias al uso de enteros de 64 bits para el direccionamiento LBA, el límite teórico de tamaño de partición y de disco en GPT asciende a $2^{64} \times 512$ bytes $= 9.44$ ZB (8 ZiB, es decir, más de nueve mil millones de terabytes). La Tabla 2 sintetiza las diferencias operativas y estructurales entre ambos esquemas.

Tabla 2: Comparación técnica exhaustiva entre los esquemas de particionamiento MBR y GPT.

Criterio técnico	**Registro de arranque maestro (MBR)**	**Tabla de particiones GUID (GPT)**
Especificación de origen	IBM PC-XT / MS-DOS (1983)	Especificación UEFI (Intel / UEFI Forum)
Ubicación de metadatos	Exclusivamente en LBA 0 (512 bytes)	LBA 0 a 33 (primaria) y LBA $N-33$ a $N-1$ (respaldo)
Particiones primarias	Máximo 4 (o 3 primarias y 1 extendida)	128 de forma estándar (ampliable si se requiere)
Particiones lógicas	Requiere encadenamiento de EBR	No aplica; todas las particiones son nativas
Capacidad máxima	$2^{32} \times 512$ B $= 2.19$ TB (2.00 TiB)	$2^{64} \times 512$ B ≈ 9.44 ZB (8 ZiB)
Campos de direccionamiento	Enteros sin signo de 32 bits	Enteros sin signo de 64 bits
Redundancia estructural	Nula; punto único de fallo absoluto	Cabecera y tabla duplicadas al final del disco
Verificación de integridad	Ninguna (salvo firma fija `0x55AA`)	Sumas de comprobación CRC32 en cabecera y tabla
Identificación de tipos	Código hexadecimal de 1 byte (0–255)	GUID universalmente único de 128 bits (16 bytes)
Identificación de volumen	Firmas de disco de 32 bits no estándar	PARTUUID único de 128 bits por partición
Compatibilidad de firmware	BIOS tradicional y UEFI bajo módulo CSM	Nativo en UEFI; requiere configuración híbrida en BIOS

1.3. Tipologías de particiones y puntos de montaje en Linux

A diferencia de los entornos operativos de la familia Windows NT, los cuales organizan el espacio de almacenamiento asignando a cada volumen formateado una letra de unidad independiente (`C:`, `D:`, `E:`) ligada a un espacio de nombres local, los sistemas basados en UNIX y Linux estructuran la totalidad de sus recursos bajo un árbol jerárquico unificado que nace de un único directorio raíz denominado `/` (Linux Foundation, 2015).

Para que los bloques de una partición física de almacenamiento resulten accesibles a los procesos y usuarios, el sistema operativo implementa el mecanismo de montaje mediante el sistema de archivos virtual (*Virtual File System*, VFS) del núcleo (Silberschatz et al., 2006, §11.9.3, pp. 398–399). El montaje consiste en vincular el superbloque y la raíz de inodos de un sistema de archivos contenido en una partición determinada a un directorio existente en el árbol jerárquico, denominado punto de montaje (*mount point*). Una vez montada la partición, cualquier operación de lectura o escritura dirigida a dicho directorio o sus subdirectorios se redirige de forma transparente por el VFS hacia el controlador del sistema de archivos residente en el dispositivo correspondiente.

En una instalación técnica avanzada orientada al arranque dual de Arch Linux junto a Windows sobre plataformas modernas UEFI/GPT, el diseño del esquema de almacenamiento exige definir con precisión las siguientes tipologías de partición y puntos de montaje (Arch Linux Project, 2026b):

1.3.1. Partición del sistema EFI (ESP)

La partición del sistema EFI (*EFI System Partition*, ESP) es un volumen mandatorio estandarizado por la especificación UEFI que actúa como puente de comunicación entre el firmware de la placa base y los gestores de arranque de los diferentes sistemas operativos instalados (Unified Extensible Firmware Interface Forum, 2024). En una tabla GPT se identifica con el GUID `C12A7328-F81F-11D2-BA4B-00A0C93EC93B` (código mnemotécnico `EF00` en utilidades como `gdisk`).

El firmware UEFI contiene controladores nativos de lectura para el sistema de archivos FAT en sus variantes FAT12, FAT16 y FAT32, siendo este último el formato estandarizado para discos duros fijos (`mkfs.fat -F 32`). En la ESP residen exclusivamente archivos ejecutables con formato PE32+ (*Portable Executable*) con extensión `.efi`, organizados bajo el subdirectorio `/EFI/`:

- En equipos donde Windows ya se encuentra preinstalado de fábrica, el instalador de Microsoft crea una ESP (típicamente de 100 MiB a 260 MiB) que aloja el binario `/EFI/Microsoft/Boot/bootmgfw.efi` correspondiente a Windows Boot Manager.
- Para configurar el arranque dual sin alterar la integridad del entorno Windows, la instalación de Arch Linux debe reutilizar dicha partición compartida o asegurar que posea suficiente espacio para albergar los cargadores correspondientes (tales como GNU GRUB en `/EFI/GRUB/grubx64.efi` o *systemd-boot*). En instalaciones nuevas donde se diseña la partición desde cero, se recomienda dimensionar la ESP entre 512 MiB y 1 GiB para admitir la colocación directa de imágenes de núcleo Linux si se opta por arranque directo mediante *EFISTUB*.
- En Arch Linux, los puntos de montaje canónicos recomendados son `/efi` (preferido cuando se busca mantener la ESP estrictamente para binarios EFI de arranque y montar `/boot` en la partición raíz) o `/boot` (habitual cuando se requiere que los núcleos y las imágenes de memoria inicial residan directamente en el espacio accesible por el firmware).

1.3.2. Partición o directorio de arranque (`/boot`)

El directorio `/boot` alberga los elementos estáticos indispensables para inicializar la ejecución del sistema operativo:

- La imagen comprimida y ejecutable del núcleo Linux (`vmlinuz-linux`).
- El archivo del sistema de archivos temporal en memoria RAM (*initial RAM filesystem*, `initramfs-linux.img` y su contraparte de contingencia `initramfs-linux-fallback.img`), el cual provee al núcleo de los módulos de controladores (almacenamiento, cifrado, sistemas de archivos raíz) necesarios antes de poder montar el volumen raíz definitivo.
- Las imágenes de microcódigo de la CPU (`intel-ucode.img` o `amd-ucode.img`) para aplicar correcciones de estabilidad y seguridad al procesador en las fases iniciales del arranque.
- Los archivos de configuración, temas y módulos dinámicos del gestor de arranque GNU GRUB (`/boot/grub/`).

En instalaciones sobre sistemas de archivos raíz estándar como ext4, `/boot` suele mantenerse simplemente como un subdirectorio ordinario dentro de la partición raíz `/`, simplificando la gestión de espacio libre. No obstante, si se utiliza cifrado de disco completo (LUKS) o sistemas de archivos con compresión y características avanzadas de instantáneas que el

gestor de arranque no pueda descifrar directamente, se justifica plenamente segregar `/boot` en una partición primaria independiente formateada en ext4 con un tamaño típico de 1 GiB a 2 GiB.

1.3.3. Partición raíz (/)

La partición raíz es el nodo cardinal del sistema operativo. Alberga la totalidad de la estructura del sistema conforme al estándar FHS (Linux Foundation, 2015): los binarios esenciales de usuario y de administración del sistema (`/usr/bin`), las bibliotecas compartidas del sistema (`/usr/lib`), los archivos de configuración estática del equipo (`/etc`), el subsistema de paquetes y variables de estado (`/var`) y los puntos de generación de sistemas de archivos virtuales dinámicos del núcleo (`/proc`, `/sys`, `/dev`).

En la tabla de particiones GPT para arquitecturas de 64 bits (x86_64), el volumen raíz se identifica de forma estandarizada mediante el GUID `4F68BCE3-E8CD-4DB1-96E7-FBCAF984B709` (código `8304` o el genérico de datos Linux `8300` en utilidades de particionado). El sistema de archivos predilecto por excelencia para la partición raíz en Arch Linux es ext4 (*Fourth Extended Filesystem*), debido a su probada estabilidad, diseño con registro en diario (*journaling*), eficiente asignación multirbloque y tolerancia a cortes de suministro eléctrico. En escenarios de ingeniería con requerimientos de instantáneas (*snapshots*) a nivel de bloques y compresión transparente sobre la marcha, se adopta con frecuencia Btrfs. Para una instalación equilibrada que contemple el sistema base, entornos de desarrollo, compiladores, cabeceras del núcleo y herramientas de administración, el dimensionamiento de la partición raíz debe situarse entre un mínimo de 30 GiB y 60 GiB cuando los datos de usuario se segregan, o el remanente total del espacio asignado a Linux en caso de esquema monovolumen.

1.3.4. Partición de datos de usuario (`/home`)

El directorio `/home` hospeda los directorios personales de cada cuenta de usuario registrada en el sistema operativo, dentro de los cuales se resguardan no solo los documentos, proyectos y descargas del usuario, sino también los archivos de configuración local específicos de cada aplicación y entorno de escritorio (comúnmente denominados *dotfiles*, situados en `~/.config/`, `~/.local/`, `~/.bashrc`).

En la arquitectura de sistemas informáticos, segregar `/home` en una partición independiente (GUID `933AC7E2-D2A2-4B10-802A-226C2C40F627`, código `8302`) aporta ventajas operativas determinantes:

- **Inmunidad ante reinstalaciones y migraciones:** Permite formatear, reparar o actualizar la partición raíz `/` de forma destructiva sin comprometer la integridad ni la configuración de los perfiles de usuario.
- **Aislamiento de cuotas y espacio:** Evita que un llenado accidental de archivos personales o descargas masivas agote los bloques disponibles para el sistema, lo cual causaría el bloqueo de demonios y servicios críticos del sistema operativo en `/var` o `/tmp`.
- **Políticas de montaje específicas:** Habilita el montaje con opciones restrictivas de seguridad, tales como `nodev` (impedir la creación de dispositivos de caracteres o bloques en el directorio personal) o `nosuid` (inhabilitar la ejecución de binarios con banderas de cambio de identificador de usuario *SetUID*).

1.3.5. Espacio y mecanismos de intercambio (*swap*)

El subsistema de memoria virtual de Linux requiere un mecanismo para gestionar el desbordamiento de la memoria física principal (RAM) mediante la técnica de intercambio de páginas (*paging/swapping*) (Silberschatz et al., 2006, §12.6, pp. 421–423). Cuando el consumo de memoria supera la capacidad instalada, o cuando el núcleo identifica páginas de memoria asignadas a procesos inactivos que no han sido referenciadas en períodos prolongados, el demonio gestor de memoria virtual (*kswapd*) extrae dichas páginas y las transfiere al espacio de intercambio, liberando marcos de página físicos para la memoria activa y las cachés de lectura/escritura de archivos.

El espacio de intercambio en Linux puede materializarse mediante dos mecanismos equivalentes:

1. **Partición de intercambio dedicada (*swap partition*):** Una partición formateada a bajo nivel con la utilidad `mkswap` (GUID GPT `0657FD6D-A4AB-43C4-84E5-0933C84B4F4F`, código `8200`). Al ser un volumen crudo sin capas intermedias de sistemas de archivos, ofrece un direccionamiento secuencial directo y rendimiento óptimo.
2. **Archivo de intercambio (*swapfile*):** Un archivo estático preasignado y contiguo ubicado dentro de un sistema de archivos regular (como ext4 o Btrfs) inicializado con `mkswap`. Provee una flexibilidad superior, puesto que permite ajustar dinámicamente su tamaño o reubicarlo sin modificar la tabla de particiones del disco.

Tabla 3: Esquema integral de particionamiento sugerido para arranque dual con Arch Linux en UEFI/GPT.

Partición	**Punto de montaje**	**Tipo FS**	**Tamaño**	**Función técnica y notas**
ESP (Windows / Linux)	`/efi` o `/boot`	FAT32	260–1024 MiB	Almacén de binarios `.efi`; se comparte la ESP existente de Windows (`EF00`).
MSR (Microsoft)	Ninguno	Crudo	16 MiB	Partición reservada de Microsoft; no tocar (`0C01`).
Windows (C:)	Ninguno	NTFS	Variable	Sistema operativo Windows y datos (`0700`).
Windows Recovery	Ninguno	NTFS	500–1000 MiB	Entorno de recuperación WinRE de fábrica (`2700`).
Linux Swap	`[SWAP]`	Swap	4–16 GiB	Paginación y soporte de hibernación ACPI S4 (`8200`).
Linux Root	`/`	ext4 / Btrfs	40–80 GiB	Raíz del sistema operativo, binarios y librerías (`8304`).
Linux Home	`/home`	ext4 / Btrfs	Remanente	Datos personales, proyectos y configuraciones (`8302`).

Un factor decisivo en la arquitectura de un equipo portátil o estación de trabajo es la compatibilidad con el estado de hibernación (suspensión a disco o estado ACPI S4). Durante la hibernación, el núcleo Linux guarda una imagen completa del estado de la memoria RAM y de los registros del procesador en el dispositivo de intercambio antes de desenergizar el hardware. Por consiguiente, si se requiere soporte de hibernación, el tamaño del espacio de intercambio debe ser al menos equivalente o ligeramente superior a la capacidad total

de memoria RAM física del equipo ($1\times$ a $1.5\times$ la RAM), asegurando que toda la memoria volátil pueda ser volcada en el medio no volátil sin truncamiento. Si no se contempla la hibernación, una asignación de 2 GiB a 4 GiB suele resultar suficiente para amortiguar picos de carga en estaciones con 16 GiB o más de RAM.

2. Arquitectura de arranque y coexistencia de sistemas operativos

El arranque de un equipo de cómputo moderno constituye una de las secuencias más críticas y rigurosas de la ingeniería de software y hardware. Lejos de ser una transición instantánea entre la alimentación eléctrica y la disponibilidad del entorno interactivo, el arranque involucra una cadena estricta de transferencias de control entre circuitos integrados de memoria no volátil, firmware de bajo nivel, cargadores intermedios y el núcleo del sistema operativo (Deitel et al., 2004, §2.4.3, pp. 76–77). Cuando dos plataformas concebidas bajo paradigmas dispares —como Microsoft Windows y Arch Linux— coexisten sobre una misma máquina física, la estabilidad global del sistema depende enteramente de la articulación precisa de estos eslabones.

2.1. Interfaces de firmware: BIOS heredado y UEFI

Al presionar el botón de encendido de la computadora, la fuente de alimentación estabiliza las líneas de tensión continua y envía la señal *Power Good* a la placa base. Esta señal libera la línea de reinicio de la CPU, forzando al procesador a inicializar sus registros en un estado predeterminado y a ejecutar la instrucción ubicada en el vector de reinicio (*reset vector*), típicamente en la dirección de memoria física `0xFFFFFFF0` mapeada a la memoria de solo lectura (*Read-Only Memory*, ROM) o memoria flash del firmware (Silberschatz et al., 2006, §12.5.2, pp. 419–420). A partir de este instante, el firmware asume la ejecución del autodiagnóstico al encendido (*Power-On Self-Test*, POST), verificando la integridad de la memoria principal, inicializando los buses del sistema (PCI Express, USB, SATA y NVMe) y preparando los dispositivos para la fase de carga.

Durante tres décadas, esta responsabilidad recayó en el sistema básico de entrada/salida tradicional (BIOS heredado o *Legacy BIOS*). Diseñado a finales de la década de 1970 para microprocesadores Intel 8086/8088 de 16 bits, el BIOS heredado arrastra compromisos arquitectónicos restrictivos:

- **Modo real de 16 bits:** La CPU opera bajo el modelo de segmentación de 16 bits, con acceso directo restringido exclusivamente al primer megabyte de memoria física (el cual se subdivide en 640 KiB de memoria convencional utilizable y 384 KiB de memoria superior reservada para vídeo y periféricos).
- **Interrupciones por software:** La comunicación entre el código de arranque y los dispositivos físicos se gestiona mediante vectores de interrupción fijos (tales como la INT 10h para primitivas de vídeo y la INT 13h para acceso a discos). Estas rutinas carecen de soporte nativo para buses serie modernos y adolecen de un rendimiento sumamente limitado al depender de llamadas síncronas bloqueantes.
- **Carga ciega de sector MBR:** Para iniciar un sistema operativo, el BIOS no interpreta sistemas de archivos ni comprende particiones. Se limita a escanear los dispositivos según el orden de arranque configurado, leer ciegamente los 512 bytes del sector

LBA 0 (el MBR) del primer dispositivo booteable, depositarlos en la dirección física fija `0x0000:0x7C00` y saltar a dicha dirección para ceder el control (Carretero Pérez et al., 2001, §7.4, pp. 377–378). Dado que 446 bytes resultan insuficientes para inicializar un núcleo moderno, los cargadores se veían forzados a implementar complejas etapas intermedias encadenadas en sectores ocultos no particionados.

Para erradicar de raíz estas limitaciones de escalabilidad y seguridad, un consorcio de líderes tecnológicos estandarizó la interfaz de firmware extensible unificada (UEFI) (Unified Extensible Firmware Interface Forum, 2024). UEFI no es una mera evolución incremental del BIOS, sino un entorno operativo completo y modular de bajo nivel, dotado de controladores de dispositivo, pila de protocolos de red, intérprete de comandos y una arquitectura desacoplada de la CPU (Imagen 2).

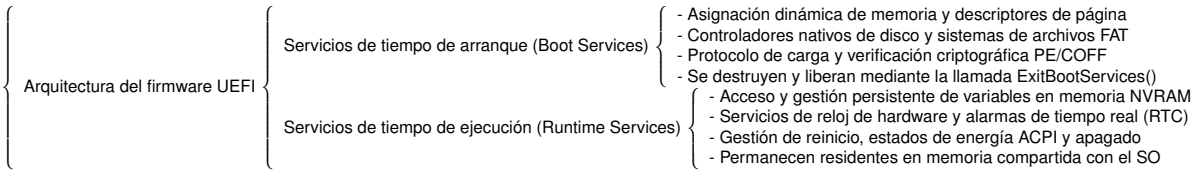


Imagen 2: Clasificación funcional de las interfaces y servicios del estándar UEFI.

El diseño de UEFI descansa en dos categorías fundamentales de interfaces de programación de aplicaciones (*Application Programming Interface*, API) que transforman la interacción entre la máquina y el sistema operativo (Unified Extensible Firmware Interface Forum, 2024):

1. **Servicios de tiempo de arranque (*Boot Services*):** Proporcionan funciones de asignación dinámica de memoria física, control de eventos y temporizadores, carga de imágenes binarias y controladores para acceso a periféricos de bloques formateados en FAT32. Los servicios de arranque están activos exclusivamente mientras el gestor de arranque toma el control. En el instante preciso en que el cargador ha cargado el núcleo Linux en la memoria principal y configurado las tablas de páginas, invoca la función canónica `ExitBootServices()`. Esta llamada notifica al firmware que el control de la máquina ha sido asumido plenamente por el núcleo del sistema operativo, liberando la totalidad de la memoria reservada por los servicios de arranque para que sea reclamada por el subsistema de memoria virtual de Linux.
2. **Servicios de tiempo de ejecución (*Runtime Services*):** A diferencia de los anteriores, estos servicios permanecen mapeados y residentes en la memoria física protegida a lo largo de toda la sesión de trabajo del sistema operativo. Entre ellos destacan la lectura y modificación de variables de firmware en la memoria de acceso aleatorio no volátil (*Non-Volatile Random-Access Memory*, NVRAM), el control de reinicio y apagado del hardware, y la sincronización del reloj de tiempo real (RTC).

La innovación más trascendental de UEFI radica en la gestión del arranque mediante el Gestor de Arranque UEFI (*UEFI Boot Manager*). En lugar de depender de código binario depositado en sectores físicos ciegos, el firmware incorpora su propio controlador de lectura de sistemas de archivos FAT y examina una serie de registros persistentes almacenados en la memoria NVRAM de la placa base (Unified Extensible Firmware Interface Forum, 2024):

- **Variables de orden de arranque (`BootOrder` y `BootNext`):** Arreglos de enteros de 16 bits que determinan la secuencia de prioridad con la que el firmware intentará ejecutar los cargadores de sistema disponibles (p. ej., `0001, 0000, 0002`).

- **Descriptores de entrada (`Boot####`):** Cada entrada individual en la NVRAM asocia una etiqueta de texto legible para el usuario (p. ej., `Windows Boot Manager` o `GRUB`) con una ruta de dispositivo UEFI (*device path*) que apunta de forma unívoca al identificador PARTUUID de la partición ESP y a la ubicación del archivo ejecutable correspondiente (p. ej., `\EFI\Microsoft\Boot\bootmgfw.efi` o `\EFI\GRUB\grubx64.efi`).

En los sistemas operativos modernos como Arch Linux, el núcleo expone la memoria NVRAM al espacio de usuario a través del pseudo-sistema de archivos `efivarfs`, montado de manera estándar bajo la jerarquía `/sys/firmware/efi/efivars/`. Esto permite a los administradores de sistemas y scripts de instalación consultar, crear, reordenar y eliminar entradas de arranque de la placa base directamente desde la línea de comandos sin necesidad de reiniciar e ingresar manualmente a la interfaz de configuración del firmware (Tabla 4).

Tabla 4: Diferencias técnicas fundamentales entre las arquitecturas BIOS tradicional y UEFI.

Parámetro técnico	**BIOS heredado (Legacy BIOS)**	**Interfaz UEFI**
Modo de CPU en arranque	Modo real de 16 bits	Modo protegido (32 bits) o modo largo (64 bits).
Memoria direccionable	Limitado a 1 MiB (640 KiB convencionales)	Acceso transparente a toda la RAM física (terabytes).
Lectura de almacenamiento	Sectores ciegos en MBR (LBA 0 en `0x7C00`)	Lectura nativa de archivos en partición ESP (FAT32).
Esquema de particiones	Master Boot Record (MBR)	GUID Partition Table (GPT) estandarizado.
Drivers del firmware	Rutinas monolíticas en ROM (interrupciones)	Módulos en C o EFI Byte Code (independientes de CPU).
Gestión de arranque	Orden estático de discos físicos	Entradas `Boot####` y variable `BootOrder` en NVRAM.
Verificación de integridad	Ninguna; susceptible a códigos maliciosos	Protocolo Secure Boot basado en certificados X.509.

2.2. Mecanismos de arranque múltiple: Windows Boot Manager y GNU GRUB

En una configuración de arranque múltiple (*dual boot*), conviven dos filosofías diametralmente opuestas para la administración de la fase inicial de ejecución. La arquitectura de arranque de la plataforma Microsoft Windows se apoya en tres elementos interconectados (Microsoft Corporation, 2024c):

1. **Windows Boot Manager (`bootmgfw.efi`):** Aplicación en formato ejecutable portátil (Portable Executable, PE32+) ubicada en la partición ESP bajo la ruta `\EFI\Microsoft\Boot\`. Es el binario que el firmware UEFI invoca según la variable `BootOrder`.
2. **Almacén de datos de configuración de arranque (BCD):** Una base de datos estructurada en formato binario tipo registro (ubicada en `\EFI\Microsoft\Boot\BCD`) que sustituyó al histórico archivo de texto `boot.ini`. El BCD almacena las entradas de arranque, descriptores de dispositivos y parámetros del núcleo para los entornos de Windows.
3. **Cargador del sistema operativo (`winload.efi`):** Invocado por Windows Boot Manager para inicializar la capa de abstracción de hardware (*Hardware Abstraction Layer*,

HAL), cargar los controladores de inicio temprano y transferir la ejecución al núcleo ejecutivo de Windows (`ntoskrnl.exe`).

Aunque el almacén BCD de Windows puede manipularse mediante la herramienta de línea de comandos `bcdedit.exe` para registrar entradas personalizadas, el gestor de arranque de Microsoft fue diseñado bajo un enfoque estrictamente monoplataforma. No cuenta con controladores para interpretar sistemas de archivos de Linux (ext4, Btrfs, XFS) ni provee mecanismos amigables para detectar de forma automatizada núcleos basados en UNIX. Además, las actualizaciones mayores acumulativas de Windows con frecuencia reescriben de forma unilateral la variable `BootOrder` de la memoria NVRAM, autodesignando a `bootmgfw.efi` como la primera opción de inicio y ocultando el acceso a otros cargadores instalados.

Por consiguiente, el estándar de ingeniería para gobernar estaciones con arranque dual radica en delegar la prioridad principal del arranque a GNU GRUB (*Grand Unified Bootloader*, versión 2) (Free Software Foundation, 2024a). En plataformas UEFI, la instalación de GRUB genera una imagen ejecutable propia (`grubx64.efi`) en el directorio `\EFI\GRUB\` de la partición ESP y registra una nueva entrada en la NVRAM de la placa base.

La arquitectura de GRUB 2 se articula como un micromicroentorno modular. Al arrancar, el binario carga dinámicamente los controladores de sistemas de archivos necesarios y procesa el archivo de configuración declarativo `/boot/grub/grub.cfg`. En distribuciones avanzadas como Arch Linux, este archivo se compila de manera reproducible ejecutando la utilidad `grub-mkconfig`, la cual evalúa las variables de `/etc/default/grub` e integra los scripts modulares del directorio `/etc/grub.d/`:

- `10_linux`: Escanea el directorio `/boot` en busca de imágenes del núcleo (`vmlinuz-linux`) e imágenes de disco de memoria inicial (`initramfs-linux.img`), generando las entradas de inicio nativas de Linux con sus respectivos identificadores de volumen UUID.
- `30_os-prober`: Invoca la utilidad `os-prober`, la cual sondea exhaustivamente todas las particiones de almacenamiento del equipo. Al identificar la presencia del cargador de Windows en la partición ESP compartida, genera automáticamente una entrada de menú que aplica el mecanismo de carga encadenada (*chainloading*). Mediante esta técnica, GRUB no intenta descifrar las estructuras internas de Windows; en su lugar, transfiere la ejecución limpia de la UEFI hacia `\EFI\Microsoft\Boot\bootmgfw.efi`, permitiendo al usuario seleccionar indistintamente entre Linux y Windows desde una única interfaz unificada.

Es indispensable enfatizar que, a partir de la versión 2.06 de GNU GRUB, los desarrolladores deshabilitaron la ejecución automática de `os-prober` por razones de seguridad, con el objetivo de prevenir la ejecución no supervisada de scripts sobre sistemas de archivos ajenos potencialmente comprometidos. Para que GRUB reconozca la instalación de Windows y genere su correspondiente entrada de menú, el usuario debe habilitar explícitamente la directiva `GRUB_DISABLE_OS_PROBER=false` en el archivo `/etc/default/grub` antes de invocar `grub-mkconfig`.

El siguiente bloque de terminal ilustra la secuencia de comandos ejecutada desde el entorno de administración para verificar las variables de firmware mediante `efibootmgr`, habilitar el sondeo de sistemas alternos y regenerar la configuración de GNU GRUB:

```
# Inspeccionar el orden de arranque y las entradas en la memoria NVRAM
efibootmgr -v

# Habilitar la detección de Windows en el archivo de configuración de GRUB
```

```
echo "GRUB_DISABLE_OS_PROBER=false»> /etc/default/grub

# Instalar el binario UEFI de GRUB en la partición ESP compartida (/boot)
grub-install -target=x86_64-efi -efi-directory=/boot -bootloader-id=GRUB

# Generar el archivo de configuración detectando Windows Boot Manager
grub-mkconfig -o /boot/grub/grub.cfg
```

2.3. Factores críticos de coexistencia: Secure Boot, Fast Startup e hibernación NTFS

La convivencia armónica y no destructiva entre Windows y Arch Linux en un mismo disco duro físico requiere neutralizar tres fuentes críticas de colisión a nivel de firmware, sistema de archivos y sincronización de hardware.

2.3.1. El protocolo Secure Boot y la cadena de confianza criptográfica

Implementado como una extensión del estándar UEFI, el arranque seguro (*Secure Boot*) tiene como finalidad impedir la inyección de código malicioso en las fases más tempranas del encendido mediante la validación de una cadena criptográfica de confianza (*chain of trust*) respaldada por firmas digitales y certificados X.509 (Unified Extensible Firmware Interface Forum, 2024). El firmware solo consiente la ejecución de controladores o cargadores UEFI si su firma digital coincide con un certificado verificado en la jerarquía de llaves almacenada en la memoria NVRAM protegida:

1. **Llave de plataforma (PK, *Platform Key*):** Es la cúspide de la jerarquía de confianza, normalmente instalada por el fabricante del equipo original (*Original Equipment Manufacturer*, OEM). Establece la propiedad administrativa de la máquina y controla las autorizaciones para modificar la llave KEK.
2. **Llave de intercambio de llaves (KEK, *Key Exchange Key*):** Gestionada por los proveedores de sistemas operativos autorizados; intermedia entre la PK y las bases de datos de firmas.
3. **Base de datos autorizada (`db`):** Almacena las firmas, certificados y sumas criptográficas SHA256 de los ejecutables EFI válidos para arrancar. Prácticamente la totalidad de los fabricantes integran de fábrica en la base de datos `db` el certificado de la Autoridad Certificadora de Microsoft (*Microsoft Corporation UEFI CA*).
4. **Base de datos revocada (`dbx`):** Lista de certificados y ejecutables comprometidos, vulnerables o sospechosos, cuya ejecución es denegada irrevocablemente por el firmware.

Debido a que el modelo de desarrollo de Arch Linux prioriza la transparencia, el autoservicio y la entrega continua (*rolling release*), sus binarios compilados de núcleo y de GRUB no vienen pre-firmados por las autoridades comerciales de Microsoft. Ante este escenario se abren dos vías de ingeniería:

- **Desactivación de Secure Boot:** Constituye el procedimiento estándar y universalmente documentado para la instalación de Arch Linux en equipos personales. Al desactivar Secure Boot desde la interfaz de configuración del firmware UEFI, el hardware ejecuta con normalidad los binarios de GRUB y de Linux sin alterar en absoluto la capacidad de arranque ni la estabilidad del sistema Windows.

- **Uso de cargadores de primera etapa (Shim / PreLoader):** Si las políticas corporativas del equipo impiden desactivar Secure Boot, es viable intercalar un binario firmado oficialmente por Microsoft denominado *Shim* (`shimx64.efi`). Este cargador inicial actúa como pasarela, administrando una base de datos local de Llaves del Propietario de la Máquina (MOK, *Machine Owner Key*) mediante la herramienta `mokutil`, permitiendo al usuario firmar sus propios núcleos y módulos de GRUB con claves privadas generadas en su instalación local.

2.3.2. Riesgo de corrupción por Windows Fast Startup e hibernación de NTFS

La amenaza técnica más severa para la integridad de los datos en un esquema de arranque dual reside en la función de inicio rápido (*Fast Startup*) de Windows. Introducida a partir de Windows 8 y activa de manera predeterminada en Windows 10 y Windows 11, esta característica implementa un apagado híbrido: al ordenar el apagado del equipo, Windows no descarga sus subsistemas ni desmonta formalmente los volúmenes NTFS. En su lugar, cierra las sesiones de los usuarios pero vuelca el estado de memoria del núcleo NT, los servicios del sistema y los controladores de dispositivo en el archivo `C:\hiberfil.sys`, congelando las estructuras de disco.

Como consecuencia de este congelamiento, los sistemas de archivos NTFS permanecen técnicamente en estado abierto o montado, con el indicador de volumen sucio (*dirty bit*) activo y transacciones de registro en diario (journaling) pendientes de confirmación. Si el usuario arranca posteriormente Arch Linux y monta las particiones de Windows para acceder a sus archivos personales en modo lectura y escritura (utilizando el controlador nativo `ntfs3` del núcleo o la biblioteca `ntfs-3g`), Linux no tiene forma de conocer las estructuras que Windows mantiene congeladas en memoria RAM dentro de `hiberfil.sys`. Cualquier escritura efectuada por Linux en la tabla maestra de archivos (MFT, *Master File Table*) o en la asignación de clusters corromperá irreversiblemente la coherencia del disco. Cuando el equipo se reinicie en Windows, este restaurará su estado de memoria congelado sobreescribiendo ciegamente el disco físico, destruyendo los datos modificados por Linux y pudiendo ocasionar fallos catastróficos de arranque en ambos sistemas.

Por tanto, es una regla técnica estricta y mandatoria desactivar por completo la hibernación y el inicio rápido de Windows antes de proceder con el particionamiento e instalación de Linux. Esto se lleva a cabo abriendo una consola con privilegios de administrador en Windows y ejecutando:

```
rem Desactivar de forma definitiva la hibernación y el Fast Startup en Windows
powercfg /h off
```

2.3.3. Sincronización del reloj de hardware en tiempo real (RTC)

La tercera causa habitual de fricción técnica proviene de la discrepancia de diseño en el manejo del reloj de tiempo real (RTC, *Real-Time Clock*) de la placa base (Deitel et al., 2004, §2.4.2, p. 76). El RTC es un circuito alimentado por una batería de botón (típicamente tipo CR2032) que mantiene el conteo de tiempo incluso cuando la computadora carece de suministro eléctrico:

- Por tradición heredada de MS-DOS, Windows asume por omisión que el RTC almacena la hora local (*local time*), aplicando directamente sobre el reloj del hardware los desfases provocados por las zonas horarias y los cambios de horario estacional.

- Por el contrario, los sistemas conformes al estándar POSIX, como Linux y la totalidad de distribuciones UNIX, configuran y leen el RTC asumiendo tiempo universal coordinado (UTC, *Coordinated Universal Time*). El núcleo calcula la hora local visualizada por el usuario en espacio de usuario a partir de las bases de datos de zonas horarias (`/usr/share/zoneinfo/`).

Cuando ambos sistemas alternan su ejecución sin una configuración coordinada, se produce un conflicto recurrente: al iniciar Windows, este lee la hora UTC del reloj creyendo que es hora local; al notar una discrepancia, la ajusta según su huso horario (p. ej., restando 6 horas para la zona centro de México, UTC-6). Cuando el usuario reinicia en Linux, el núcleo asume que el RTC sigue en UTC, volviendo a restar 6 horas al presentar la hora local. Este desajuste cíclico corrompe las marcas de tiempo de los archivos (*timestamps*), invalida certificados SSL/TLS con comprobaciones de expiración y altera los registros del sistema.

Para resolver de manera definitiva este conflicto, la solución de ingeniería recomendada consiste en forzar a Windows a operar su reloj de hardware en estándar UTC, garantizando paridad total con Linux sin comprometer la sincronización horaria. Esto se implementa agregando el valor `RealTimeIsUniversal` en el Registro de Windows:

```
rem Configurar Windows para interpretar el reloj de hardware (RTC) en UTC
reg add "HKLM\SYSTEM\CurrentControlSet\Control\TimeZoneInformation"^
   /v RealTimeIsUniversal /t REG_DWORD /d 1 /f
```

En Linux, la verificación del estado de sincronización del reloj de hardware y del demonio de tiempo en red se realiza mediante la utilidad `timedatectl`:

```
# Verificar que el reloj del sistema opere con el RTC en tiempo UTC
timedatectl status
```

3. Requerimientos de hardware y preparación del entorno

La instalación controlada de un sistema operativo en coexistencia con otra plataforma exige una evaluación rigurosa del hardware anfitrión y la implementación de protocolos de mitigación de riesgos. A diferencia de las distribuciones orientadas al usuario final que integran instaladores automáticos con detección heurística de recursos, Arch Linux delega la totalidad de las decisiones operativas en el ingeniero o administrador de sistemas (Arch Linux Project, 2026b). En este esquema de responsabilidad compartida, el desconocimiento de las capacidades de la máquina o la omisión de respaldos preventivos puede derivar en la inaccesibilidad irreversible de los volúmenes preexistentes.

3.1. Especificaciones técnicas mínimas y arquitectura de soporte en Arch Linux

La viabilidad técnica de una instalación moderna descansa en la concordancia entre los requerimientos del núcleo Linux, los subsistemas de gestión de memoria y el conjunto de instrucciones implementado por el procesador (McHoes & Flynn, 2011, §1.2, pp. 4–6). Arch Linux opera bajo un modelo de desarrollo de liberación continua (rolling release) enfocado exclusivamente en la arquitectura x86_64 (Intel 64 y AMD64). El soporte para la arquitectura heredada i686 (32 bits) fue descontinuado formalmente en 2017, consolidando

un estándar operativo optimizado para registros de 64 bits y direccionamiento de memoria física extendido.

A nivel de microarquitectura, las compilaciones de paquetes oficiales de Arch Linux establecen como línea base el nivel de conjunto de instrucciones x86-64-v1, lo que asegura compatibilidad universal con procesadores comercializados a partir de mediados de la década de 2000 (incorporando extensiones SSE, SSE2, SSE3 y la instrucción `CMPXCHG16B` para operaciones atómicas de sincronización en memoria). No obstante, para cargas de trabajo científicas o entornos interactivos exigentes, el núcleo explota de forma dinámica los niveles x86-64-v2 y x86-64-v3 presentes en microprocesadores contemporáneos (añadiendo instrucciones AVX, AVX2, FMA y POPCNT), optimizando el rendimiento de librerías criptográficas y algoritmos de compresión.

En lo relativo al subsistema de memoria principal (RAM), es necesario distinguir entre los requerimientos para inicializar el medio de instalación en vivo (*live environment*) y los requerimientos funcionales para el sistema operativo en régimen de producción:

- **Memoria en el medio de instalación:** La imagen viva de Arch Linux requiere un umbral mínimo de 512 MiB de memoria física disponible para inicializar el núcleo y descomprimir el sistema de archivos raíz en memoria RAM (*ramfs* o `zram`). Sin embargo, para ejecutar con solvencia el gestor de paquetes Pacman y compilar dinámicamente las imágenes de memoria inicial (`initramfs`) durante la instalación, se requiere disponer de al menos 1 GiB a 2 GiB de memoria RAM libre. Con memorias inferiores a 1 GiB, la concurrencia entre los demonios de red y los procesos de extracción puede detonar el mecanismo asesino por falta de memoria del núcleo (*Out-Of-Memory Killer*, OOM Killer), abortando la transacción y dejando el sistema de archivos de destino en un estado inconsistente.
- **Memoria para estación de trabajo con arranque dual:** En un equipo destinado a propósitos de ingeniería, desarrollo de software o soporte a servidores, se recomienda una asignación mínima de 8 GiB a 16 GiB de memoria RAM. Esta capacidad garantiza la ejecución simultánea de entornos de escritorio modernos (KDE Plasma, GNOME o gestores de ventanas bajo el protocolo Wayland), herramientas de contenedorización (Docker, Podman) y compiladores, mitigando la necesidad de incurrir en paginación intensiva hacia el dispositivo de almacenamiento masivo.

En el ámbito del almacenamiento secundario, las especificaciones dependen directamente de la compartimentación de volúmenes analizada en la Sección 1.3. Aunque una instalación elemental de Arch Linux en modo consola consume únicamente entre 2 GiB y 3 GiB en disco, la asignación de espacio útil debe contemplar el crecimiento operativo del sistema:

- **Partición raíz (`/`):** Se recomienda dimensionar la partición raíz en un rango de 40 GiB a 80 GiB. Este espacio alberga los binarios ejecutables (`/usr/bin`), las librerías compartidas (`/usr/lib`), los registros persistentes del sistema gestionados por *systemd-journald* (`/var/log`) y la caché de paquetes de Pacman (`/var/cache/pacman/pkg/`). Dado que Arch Linux no elimina de forma automática los paquetes descargados tras cada actualización, una partición raíz constreñida a 15 GiB o 20 GiB puede saturarse prematuramente si el usuario no instrumenta rutinas periódicas de limpieza de caché.
- **Partición de usuarios (`/home`):** Debe acaparar el espacio remanente disponible en la cuota asignada a Linux en el disco (idealmente 50 GiB o más), proveyendo aislamiento para código fuente, entornos virtuales y documentos personales.
- **Partición del sistema EFI (ESP):** Cuando se comparte la partición ESP preexistente generada por Windows (típicamente de 100 MiB a 260 MiB), el usuario debe verificar

que disponga de al menos 50 MiB a 100 MiB de espacio libre para alojar los binarios de GRUB y, opcionalmente, las imágenes del núcleo Linux si se opta por un esquema de montaje directo en `/boot`.

Respecto a la placa base y el firmware, la instalación de arranque dual exige soporte nativo para UEFI de 64 bits. Es imperativo ingresar a la interfaz de configuración del firmware (Setup UEFI) previo a la instalación y desactivar el módulo de soporte de compatibilidad (*Compatibility Support Module*, CSM), el cual emula el comportamiento de una BIOS heredada. Mantener habilitado CSM puede inducir al medio de arranque USB a inicializarse en modo de 16 bits; si el instalador arranca bajo BIOS, el núcleo Linux no tendrá acceso al subsistema de variables de firmware `efivarfs`, impidiendo irrevocablemente la instalación del gestor de arranque UEFI y la configuración de entradas en la NVRAM.

Finalmente, a diferencia de otras distribuciones de Linux que empaquetan instantáneas completas de paquetes en imágenes ISO de 4 GiB o más, el medio de instalación oficial de Arch Linux es estrictamente un instalador por red (*net-install*) de volumen reducido (1 GiB). La imagen viva contiene únicamente las herramientas esenciales de diagnóstico y particionamiento; los paquetes definitivos del sistema operativo base (`base`, `linux`, `linux-firmware`) se descargan en tiempo real desde los servidores réplica (*mirrors*) oficiales durante la ejecución del comando `pacstrap`. En consecuencia, el equipo de cómputo debe contar con una interfaz de red operativa reconocida por el núcleo: ya sea una conexión cableada Ethernet (gestionada de forma automática vía DHCP) o una interfaz inalámbrica Wi-Fi operable mediante el demonio de red `iwd` (Tabla 5).

Tabla 5: Matriz comparativa de especificaciones técnicas para Arch Linux en arranque dual.

Componente	**Requerimiento mínimo absoluto**	**Configuración recomendada (Ingeniería)**
Arquitectura CPU	x86_64 (x86-64-v1, SSE3, `CMPXCHG16B`)	x86_64 multinúcleo con extensiones x86-64-v3 (AVX2).
Memoria RAM	512 MiB (modo consola estricto)	8 GiB a 16 GiB (entorno gráfico, compilación y Docker).
Almacenamiento (Raíz)	2 GiB (sistema base mínimo)	40 GiB a 80 GiB (ext4/Btrfs, paquetes y registros).
Almacenamiento (Home)	Espacio remanente en disco	50 GiB a 200+ GiB (proyectos y datos de usuario).
Espacio de intercambio	2 GiB (sin hibernación)	$1\times$ a $1.5\times$ la RAM física (soporte de ACPI S4).
Firmware de placa base	UEFI nativo de 64 bits (CSM desactivado)	UEFI con firmware actualizado y Secure Boot gestionado.
Conectividad a red	Conexión activa a Internet (Ethernet o Wi-Fi)	Conexión de banda ancha estable para descarga de réplicas.
Unidad USB de arranque	Dispositivo USB de 2 GiB	Dispositivo USB 3.0/3.1 de 8 GiB de alta velocidad.

3.2. Diagnóstico, verificación de compatibilidad e integridad criptográfica de la ISO

El proceso de obtención del medio de instalación de Arch Linux debe regirse por estrictos principios de seguridad de la información. Debido a que el software de arranque posee

privilegios de ejecución sin restricciones sobre el hardware anfitrión (anillo 0 del procesador o modo supervisor), la ejecución inadvertida de una imagen adulterada, dañada durante la descarga o intervenida por un actor malicioso mediante un ataque de intermediario (*man-in-the-middle*, MITM) comprometería la totalidad del equipo, incluyendo los datos del sistema Windows preexistente.

Para mitigar esta vulnerabilidad, el consorcio de desarrollo de Arch Linux publica las imágenes de disco en una red mundial de réplicas acompañadas de dos mecanismos complementarios de verificación criptográfica (Arch Linux Project, 2026a):

1. **Verificación de integridad mediante suma de comprobación (SHA-256):** Permite certificar que la imagen descargada es matemáticamente idéntica, bit a bit, a la compilada originalmente por los desarrolladores, descartando corrupciones accidentales derivadas de interrupciones de red, errores de paridad en el almacenamiento o descargas incompletas. La función hash criptográfica SHA-256 genera una huella unidireccional de 256 bits (representada como una cadena hexadecimal de 64 caracteres). Gracias a la propiedad del efecto avalancha (*avalanche effect*), la modificación o inversión de un único bit en el archivo ISO de más de un gigabyte altera completamente el valor de la huella resultante.
2. **Verificación de autenticidad mediante OpenPGP (GNU Privacy Guard, GPG):** Aunque una suma SHA-256 valida la integridad del archivo frente a daños accidentales, no previene que un atacante suplante simultáneamente la imagen y el archivo de suma en un servidor réplica vulnerado. Para garantizar que la imagen proviene auténticamente del equipo oficial de ingeniería de Arch Linux, cada versión incluye un archivo de firma digital separada (*detached signature*) con extensión `.iso.sig`. Mediante el protocolo OpenPGP (RFC 4880), el usuario valida la firma digital utilizando la llave pública del desarrollador responsable de la compilación de la imagen, cuya validez se encuentra respaldada por la red de confianza (*web of trust*) y las llaves maestras de firma del proyecto.

Para realizar estas comprobaciones en una estación de trabajo basada en Linux o en un entorno de consola compatible, se descargan tanto la imagen ISO como sus archivos auxiliares de firma y suma. El siguiente bloque ilustra la verificación de integridad empleando la herramienta `sha256sum` y la comprobación de autenticidad mediante `gpg`:

```
# 1. Calcular la huella SHA-256 y compararla contra el registro oficial
sha256sum -c archlinux-2026.09.01-x86_64.iso.sha256
# Salida esperada: archlinux-2026.09.01-x86_64.iso: OK

# 2. Recuperar la llave pública del desarrollador y verificar la firma PGP
gpg -keyserver-options auto-key-retrieve -verify ^
   archlinux-2026.09.01-x86_64.iso.sig archlinux-2026.09.01-x86_64.iso
```

En la salida de `gpg`, la presencia de la frase `Firma correcta` (*Good signature*) confirma que el binario no ha sido manipulado con posterioridad a su firma por el desarrollador. La advertencia complementaria relativa a la ausencia de una firma de confianza certificada simplemente denota que la clave del desarrollador no ha sido validada localmente por el usuario dentro de su propio anillo de llaves (*keyring*), lo cual resulta natural y seguro siempre que la huella digital (*key fingerprint*) coincida con las huellas de publicación oficiales documentadas en el portal institucional del proyecto.

Una vez creado el medio USB de arranque e inicializada la computadora con él, el usuario accede al intérprete de comandos interactivo de la consola de Arch Linux (identificado con el usuario privilegiado `root@archiso`). Antes de proceder con la modificación de la tabla

de particiones, es un deber técnico ineludible ejecutar un protocolo de diagnóstico sobre los subsistemas de la máquina para validar la compatibilidad del entorno:

1. **Verificación del modo de inicio del firmware:** Se comprueba la disponibilidad del pseudo-sistema de archivos `efivarfs`. Si el comando de verificación lista los descriptores de variables, la máquina arrancó con éxito en modo UEFI nativo de 64 bits. Si el directorio no existe o el sistema devuelve un error, la máquina arrancó bajo BIOS heredado, debiendo reiniciarse inmediatamente para corregir el orden de arranque en el firmware antes de realizar cualquier cambio en el disco:

```
# Verificar si el kernel cargó las interfaces UEFI de 64 bits
ls /sys/firmware/efi/efivars
```

2. **Auditoría de arquitectura del procesador:** Mediante la herramienta `lscpu`, se constata que la CPU soporte el modo de operación de 64 bits (`CPU op-mode(s): 32-bit, 64-bit`) y se inspeccionan las banderas de virtualización por hardware (`VT-x` para Intel o `AMD-V` para AMD), indispensables para herramientas de virtualización y seguridad en ingeniería.
3. **Inspección de memoria física:** Se invoca `free -h` para verificar que el núcleo reconozca la totalidad de los módulos de RAM instalados y evaluar el uso de memoria en el entorno temporal en vivo.
4. **Reconocimiento de topología de almacenamiento:** Mediante la utilidad `lsblk -f`, se inventarían todos los dispositivos de almacenamiento no volátiles reconocidos por el subsistema de bloques del núcleo (tales como unidades de estado sólido NVMe en `/dev/nvme0n1` o discos SATA en `/dev/sda`), sus particiones existentes, los sistemas de archivos detectados (NTFS, FAT32) y sus identificadores de volumen UUID.

3.3. Estrategias de respaldo y mitigación de riesgos de pérdida de datos

La coexistencia de dos sistemas operativos en un mismo disco rígido físico introduce una frontera compartida de alta sensibilidad operativa. El procedimiento de habilitar espacio no asignado para Arch Linux exige reducir el tamaño de la partición de Windows (C:) y alterar los registros de la tabla GPT del disco. En este proceso, fallos transitorios en el suministro eléctrico, errores humanos en la selección de índices de partición o corrupciones de metadatos en el sistema de archivos NTFS pueden desencadenar la destrucción total de las particiones preexistentes (Silberschatz et al., 2006, §12.7, pp. 424–426). Por tanto, la disciplina de ingeniería exige diseñar e instrumentar una jerarquía defensiva de mitigación y respaldo previa a cualquier operación de escritura (Imagen 3).

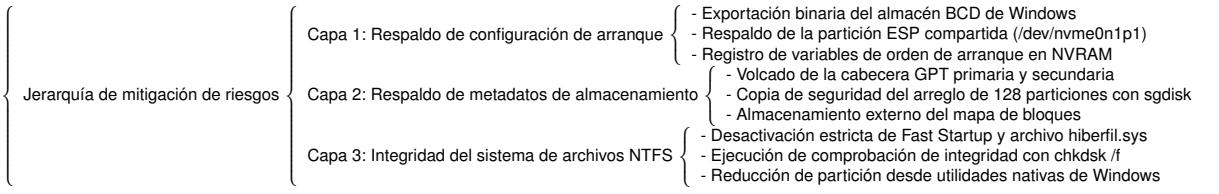


Imagen 3: Protocolo estructurado en tres capas para la mitigación de riesgos y respaldo previo a la instalación.

El protocolo de seguridad se articula secuencialmente a través de tres niveles defensivos:

3.3.1. Capa 1: Respaldo del almacén BCD y de la partición ESP

El almacén BCD (*Boot Configuration Data*) contiene la definición de todas las rutas de arranque que gobiernan el inicio de Windows (Microsoft Corporation, 2024a). Si durante la instalación de GRUB se sobrescribiera por error la configuración de la ESP o se dañara el directorio `\EFI\Microsoft\`, Windows quedará imposibilitado para iniciar. Para salvaguardar esta configuración, se abre una consola de comandos con privilegios elevados en Windows previo a iniciar la instalación y se ejecuta la exportación del BCD hacia un medio de almacenamiento externo:

```
rem Exportar el almacén BCD de Windows hacia una unidad de almacenamiento externa
bcdedit /export D:\Windows_BCD_Backup\BCD_Original
```

Adicionalmente, desde el medio en vivo de Arch Linux, se puede generar una imagen forense exacta a nivel de bloques de la partición ESP compartida utilizando la utilidad clásica `dd` (Stallings, 2018, §11.7, pp. 532–535). Este comando captura tanto el sistema de archivos FAT32 como los ejecutables binarios de Windows Boot Manager en un archivo de imagen recuperable:

```
# Crear una imagen cruda de la partición ESP (suponiendo que es nvme0n1p1)
dd if=/dev/nvme0n1p1 of=/mnt/externo/esp_backup.img bs=64K status=progress
```

3.3.2. Capa 2: Respaldo de metadatos de particionamiento con sgdisk

La manipulación interactiva de herramientas de particionamiento encierra el riesgo latente de cometer una errata tipográfica y eliminar inadvertidamente una partición protegida de Windows o la partición de recuperación del fabricante. Para garantizar una restitución instantánea de la geometría de disco ante cualquier contingencia, la utilidad `sgdisk` (componente de *GPT fdisk*) permite crear una copia de seguridad binaria que abarca la cabecera GPT primaria, la cabecera secundaria de respaldo y el arreglo completo de 128 descriptores de partición:

```
# Realizar un respaldo binario exhaustivo de la tabla de particiones GPT
sgdisk -backup=/mnt/externo/gpt_nvme0n1_backup.bin /dev/nvme0n1
```

En el hipotético escenario de una destrucción accidental de la tabla de particiones, la restauración atómica se efectúa de manera inmediata invocando:

```
# Restaurar la estructura GPT completa a partir de la copia binaria
sgdisk -load-backup=/mnt/externo/gpt_nvme0n1_backup.bin /dev/nvme0n1
```

3.3.3. Capa 3: Integridad de NTFS y reducción del volumen desde Windows

Una de las decisiones arquitectónicas más críticas en la preparación del entorno consiste en definir desde qué sistema operativo debe ejecutarse la reducción del volumen de Windows para liberar el espacio donde se instalará Arch Linux.

Aunque herramientas de particionamiento ejecutadas desde Linux (como GParted o parted) poseen capacidades teóricas para redimensionar sistemas de archivos NTFS apoyándose en la librería `ntfs-3g`, recurrir a utilidades externas para contraer un volumen de Windows activo entraña un riesgo severo de corrupción estructural. El sistema de archivos NTFS es un sistema propietario altamente complejo que aloja estructuras internas de metadatos fijas, tales como la tabla maestra de archivos (MFT), archivos de paginación (`pagefile.sys`), volcados de memoria y copias de seguridad de instantáneas de volumen (*Volume Shadow Copies*). Estas estructuras son marcadas frecuentemente como archivos inamovibles (*unmovable files*) por el controlador del núcleo de Windows.

Si una utilidad externa intenta reubicar estos sectores ciegos sin el concurso del subsistema de E/S de Windows, puede corromper los punteros de registro en diario (journaling) o provocar que el sistema de archivos sea declarado inconsistente durante el siguiente inicio, obligando a Windows a ejecutar una reparación destructiva (`chkdsk`) que suele truncar o eliminar los datos de usuario desplazados.

Por consiguiente, la mejor práctica de ingeniería estipula que la reducción del volumen de disco C: debe ejecutarse obligatoriamente desde el entorno interactivo del propio Windows, empleando el Administrador de Discos gráfico (`diskmgmt.msc`) o la consola `diskpart.exe`. El controlador de almacenamiento de Windows es la única entidad que conoce con precisión matemática los límites físicos donde residen los metadatos inamovibles, garantizando que el límite de compresión no sobrepase los límites de seguridad y dejando un espacio libre no asignado (*unallocated space*) limpio y continuo sobre el cual las utilidades de particionamiento de Linux podrán operar posteriormente de forma segura y sin riesgo de colisión.

4. Herramientas de particionamiento y gestión de almacenamiento

La manipulación de estructuras de almacenamiento secundario representa una de las tareas de mayor trascendencia en la administración de sistemas operativos. Una modificación defectuosa en la tabla de particiones o un cálculo erróneo en los desplazamientos de sectores puede ocasionar la superposición de volúmenes o la pérdida total de sistemas de archivos enteros (Silberschatz et al., 2006, §12.5.2, pp. 419–420). Por ello, el ingeniero debe dominar el funcionamiento interno, las limitaciones de seguridad y los modelos de confirmación de las herramientas de gestión de almacenamiento, tanto en el ecosistema Windows como en las utilidades de bajo nivel de GNU/Linux.

4.1. Herramientas nativas de Windows: Administrador de discos y DiskPart

Dentro del sistema operativo Microsoft Windows, la administración del almacenamiento secundario está gobernada por la arquitectura del Servicio de Discos Virtuales (*Virtual Disk Service*, VDS), el cual expone interfaces unificadas de administración para discos básicos, discos dinámicos y volúmenes montados. El sistema provee dos interfaces operativas complementarias:

4.1.1. El Administrador de discos gráfico (diskmgmt.msc)

El Administrador de discos (*Disk Management*) es un complemento de la Consola de Administración de Microsoft (*Microsoft Management Console*, MMC). Proporciona una representación gráfica bidimensional de las unidades físicas detectadas, mostrando de forma visual la correlación entre discos físicos, particiones primarias, volúmenes con letra de unidad asignada y espacios no asignados.

A través de su menú contextual, la herramienta ofrece la función “Reducir volumen” (*Shrink Volume*). Al invocar esta acción, el Administrador de discos interactúa directamente con el controlador del sistema de archivos NTFS para determinar el tamaño máximo que puede liberarse de forma segura. Sin embargo, esta interfaz gráfica adolece de una limitación arquitectónica severa: no permite al usuario indicar qué estructuras reubicar. Si un archivo inamovible (como un registro de eventos del sistema, el archivo de intercambio `pagefile.sys` o un punto de restauración de instantáneas de volumen) reside en la mitad física del volumen, la utilidad truncará el límite máximo de reducción a dicha frontera, impidiendo liberar el espacio contiguo necesario a menos que se recurra a utilidades de línea de comandos avanzadas.

4.1.2. La utilidad de línea de comandos DiskPart

Para superar las restricciones de la interfaz gráfica y permitir la automatización determinista mediante scripts de consola, Windows incorpora la utilidad de bajo nivel `diskpart.exe` (Microsoft Corporation, 2024b). A diferencia de los comandos convencionales de Windows, `diskpart` implementa un intérprete interactivo propio estructurado bajo una jerarquía estricta de objetos de almacenamiento:

$$\text{DISCO} \longrightarrow \text{PARTICIÓN} \longrightarrow \text{VOLUMEN} \tag{3}$$

El modelo operativo de DiskPart exige enfocar (*focus*) explícitamente el objeto sobre el cual se ejecutarán las instrucciones. Cualquier comando destructivo o de modificación afectará de manera irrevocable al objeto actualmente seleccionado.

Para ejecutar una contracción controlada del volumen de Windows sin arriesgar la consistencia del sistema de archivos, el protocolo técnico en consola exige inspeccionar los límites de compresión mediante la consulta `shrink querymax` antes de ordenar la reducción del tamaño del volumen. El siguiente bloque de terminal documenta la secuencia completa de comandos ejecutada desde una consola de comandos con privilegios administrativos en Windows:

```
rem Iniciar el intérprete interactivo de bajo nivel
diskpart

DISKPART> list disk
DISKPART> select disk 0
DISKPART> list partition
DISKPART> select partition 3

rem Consultar la cantidad máxima de bytes que pueden reducirse con seguridad
DISKPART> shrink querymax

rem Reducir el volumen en 100 GiB (102400 MiB), exigiendo al menos 50 GiB libres
DISKPART> shrink desired=102400 minimum=51200
```

```
rem Verificar la creación del espacio no asignado contiguo
DISKPART> list partition
DISKPART> exit
```

El parámetro `desired` especifica la cantidad objetivo de espacio a liberar en megabytes, mientras que `minimum` fija un umbral de tolerancia por si la fragmentación del volumen impide alcanzar la cuota deseada. La gran ventaja técnica de ejecutar esta operación mediante DiskPart radica en que el subsistema NTFS se encarga de reubicar internamente los bloques de datos ordinarios hacia los primeros sectores del volumen, actualizando la tabla maestra de archivos (MFT) de forma atómica y dejando un segmento de espacio no asignado (unallocated space) perfectamente alineado al final de la partición.

4.2. Utilidades de línea de comandos en Linux: fdisk, gdisk y parted

Al arrancar el entorno en vivo de Arch Linux, el núcleo registra las unidades de almacenamiento como nodos de dispositivo de bloques bajo el directorio `/dev/`. En este punto, el administrador dispone de una gama de utilidades de terminal para manipular la tabla GPT (Stallings, 2018, §11.5, pp. 525–527). Aunque todas interactúan en última instancia con las estructuras de disco mediante llamadas al sistema del núcleo (tales como `ioctl(fd, BLKRRPART)` para forzar al núcleo a releer la tabla de particiones), difieren sustancialmente en su filosofía de diseño, su modelo de confirmación en memoria y su compatibilidad con estándares modernos.

4.2.1. util-linux fdisk

La utilidad `fdisk` constituye la herramienta de particionamiento más emblemática y tradicional del entorno UNIX/Linux. Históricamente concebida para la gestión de tablas MBR, las versiones contemporáneas integradas en el paquete `util-linux` incorporan soporte completo para tablas de particiones GPT.

`fdisk` opera mediante un menú interactivo conducido por letras mnemotécnicas simples (p. ej., `m` para mostrar la ayuda, `p` para imprimir el estado actual de las particiones, `n` para crear una partición nueva, `t` para alterar el código de tipo, `d` para eliminar un registro y `w` para escribir los cambios a disco). Su mayor virtud radica en la automatización de la alineación de sectores: por defecto, calcula los sectores iniciales de cada nueva partición en múltiplos de 2048 sectores (1 MiB), previniendo desalineaciones con las páginas físicas de las unidades de estado sólido (SSD).

No obstante, su manejo de los tipos de partición bajo GPT puede resultar engorroso, pues requiere seleccionar los identificadores a partir de listas extensas o cadenas GUID completas en lugar de códigos abreviados optimizados.

4.2.2. GPT fdisk (gdisk)

Desarrollada por Roderick W. Smith específicamente para solventar las deficiencias de `fdisk` frente al estándar UEFI, `gdisk` (GPT fdisk) es la herramienta de referencia técnica recomendada para la administración de tablas GPT en estaciones de trabajo modernas (Smith, 2022).

La arquitectura de `gdisk` se distingue por tres propiedades fundamentales:

1. **Modelo de confirmación en memoria volátil:** Al abrir un dispositivo (p. ej., `gdisk /dev/nvme0n1`), la utilidad lee las cabeceras GPT primaria y secundaria del disco y carga una réplica exacta de las estructuras en la memoria RAM del proceso. Todas las operaciones de particionamiento subsecuentes (creación, ajuste de límites, cambio de nombres y asignación de atributos) se ejecutan exclusivamente sobre la memoria volátil. El medio de almacenamiento físico no experimenta alteración alguna hasta que el usuario emite deliberadamente la instrucción `w` (*write*). Si se detecta un error o se desea abortar el procedimiento, basta con ingresar `q` (*quit*) para salir sin modificar el soporte físico.
2. **Códigos de tipo hexadecimales de cuatro dígitos:** Para simplificar la especificación de los engorrosos identificadores GUID de 128 bits estandarizados por UEFI, `gdisk` implementa un mapa de traducción transparente. Así, el tipo de sistema de archivos raíz de Linux en x86_64 (`4F68BCE3-E8CD-4DB1-96E7-FBCAF984B709`) se especifica de forma compacta mediante el código `8304`; la partición del sistema EFI (`C12A7328-F81F-11D2-BA4B-00A0C93EC93B`) se invoca con `ef00`; la partición de swap con `8200`; y la partición de datos de usuario (`/home`) con `8302`.
3. **Menú de recuperación y transformación estructural:** A través de su submenú de recuperación y transformación (invocado mediante la tecla `r`), `gdisk` permite reconstruir una cabecera primaria dañada a partir del respaldo secundario ubicado al final del disco, verificar la paridad CRC32 y sintetizar un MBR de protección (PMBR) higiénico sin perturbar los datos de usuario.

El siguiente bloque de terminal documenta la sintaxis interactiva de creación de la partición raíz de Linux empleando `gdisk`:

```
# Invocar gdisk sobre la unidad de estado sólido NVMe
gdisk /dev/nvme0n1

Command (? for help): n
Partition number (4-128, default 4): 4
First sector (34-1000215182, default = 800215040) or {+-}size{KMGTP}: [Enter]
Last sector or {+-}size{KMGTP}: +60G
Current type is 8300 (Linux filesystem)
Hex code or GUID (L to show codes, Enter = 8300): 8304
Changed type of partition to 'Linux x86-64 root (/)'

Command (? for help): p
Command (? for help): w
Do you want to proceed? (Y/N): Y
```

4.2.3. GNU Parted

A diferencia de `fdisk` y `gdisk`, concebidas bajo paradigmas de menús interactivos, GNU Parted (`parted`) es una herramienta diseñada con capacidades avanzadas de scripting y manipulación directa desde la línea de órdenes (Free Software Foundation, 2024b).

Parted se apoya en la biblioteca compartida `libparted`, la cual implementa controladores para reconocer tanto etiquetas de disco (GPT, MS-DOS, Sun, Mac) como parámetros internos de sistemas de archivos. Su característica distintiva es la admisión de unidades de medida explícitas en sus argumentos: el usuario puede definir desplazamientos en sectores (`s`), bytes (`B`), kibibytes (`kiB`), mebibytes (`MiB`), gibibytes (`GiB`) o porcentajes del disco (`%`).

Sin embargo, GNU Parted demanda una cautela extrema por parte del operador debi-

do a su modelo de confirmación: **Parted no retiene las modificaciones en un búfer de memoria volátil**. Cualquier orden ejecutada (como `mkpart` o `rm`) se escribe de forma instantánea sobre la tabla de particiones del disco físico, sin confirmación posterior ni posibilidad de revertir la acción con un comando de cancelación. Por esta razón, su empleo en modo interactivo suele ser desaconsejado para operadores noveles, reservándose predominantemente para scripts desatendidos de aprovisionamiento automatizado (Tabla 6).

Tabla 6: Matriz comparativa de herramientas de particionamiento en línea de comandos.

Herramienta	**Esquemas soportados**	**Modelo de confirmación**	**Alineación de sectores**	**Ámbito de aplicación idóneo**
DiskPart	MBR, GPT	Inmediato en disco	Automática por VDS	Reducción segura de NTFS en entorno Windows.
util-linux fdisk	MBR, GPT	En memoria (escritura final con `w`)	Automática a 2048 sectores (1 MiB)	Edición rápida e interactiva en consola Linux.
GPT fdisk (gdisk)	GPT exclusivo	En memoria (escritura final con `w`)	Automática a 1 MiB con tipos hexadecimales	Estándar para UEFI/GPT, diagnóstico y recuperación.
GNU Parted	MBR, GPT y otros	Inmediato en disco (sin confirmación)	Configurable (`optimal` o explícita)	Automatización mediante scripts e instaladores desatendidos.

4.3. Entornos gráficos y herramientas en vivo: GParted

En la administración de estaciones de trabajo, el Editor de Particiones de GNOME (*GNOME Partition Editor*, GParted) constituye la herramienta gráfica más ampliamente extendida para la reestructuración de almacenamiento en entornos GNU/Linux (GParted Project, 2024). Aunque en la instalación estándar de Arch Linux su uso se encuentra restringido debido a que la imagen oficial prescinde de entorno gráfico, su estudio resulta esencial para comprender los mecanismos visuales y transaccionales de manipulación de volúmenes.

4.3.1. Arquitectura modular de GParted

GParted no implementa de forma nativa los algoritmos de bajo nivel para formatear o alterar cada sistema de archivos en particular. En su lugar, opera como un front-end gráfico en GTK que se articula modularmente sobre la infraestructura del sistema operativo anfitrión (Imagen 4):

- **Capa de presentación y transacciones (GTK / GParted Core):** Administra la interfaz visual con el usuario, coordina la cola de operaciones diferidas y verifica la consistencia de las solicitudes antes de transmitirlas a las capas inferiores.
- **Capa de particionamiento (`libparted`):** Provee la biblioteca programática estandarizada de GNU Parted para consultar la geometría del disco, calcular sectores de inicio y fin, y reescribir los registros de la tabla GPT.
- **Capa de utilidades de sistemas de archivos (*userland utilities*):** Cada acción específica sobre los datos de un volumen delega su ejecución en las herramientas nativas

del sistema de archivos correspondiente: `e2fsprogs` (`resize2fs`, `e2fsck`) para volúmenes ext2/ext3/ext4; `dosfstools` (`fatlabel`, `fsck.vfat`) para la partición ESP en FAT32; `ntfs-3g` y `ntfsprogs` (`ntfsresize`) para volúmenes de Windows; y `btrfs-progs` para sistemas avanzados Btrfs.

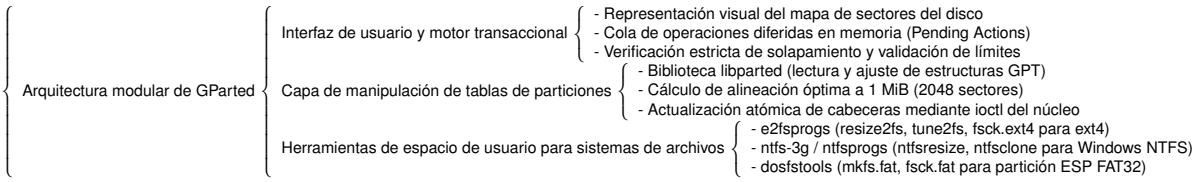


Imagen 4: Flujo arquitectónico de ejecución de operaciones de almacenamiento en GParted y su interacción modular.

4.3.2. El modelo de transacciones en cola diferida

La propiedad arquitectónica más valiosa de GParted radica en su cola de operaciones pendientes (*pending operations queue*). Cuando el usuario aplica una modificación en la interfaz gráfica (como reducir el tamaño de un volumen o crear una partición nueva), la herramienta no ejecuta la acción en el disco de manera inmediata. En su lugar, registra la instrucción en una cola de memoria secuencial y actualiza la representación visual de la unidad para reflejar cómo quedará el soporte tras la aplicación de los cambios.

Este modelo confiere una salvaguarda de seguridad crítica: el usuario puede revisar minuciosamente la secuencia acumulada de pasos, verificar las dimensiones exactas resultantes y, si advierte un desajuste o riesgo imprevisto, retroceder paso a paso o vaciar por completo la cola sin que el hardware haya recibido una sola instrucción de escritura. Solo cuando se presiona el botón “Aplicar todas las operaciones” (*Apply All Operations*), GParted adquiere bloqueos exclusivos sobre los dispositivos y ejecuta las transacciones en orden serializado.

4.3.3. Mecánica física de reubicación y redimensionamiento de volúmenes

Comprender la física del redimensionamiento de particiones es vital para prevenir desastres de almacenamiento. En GParted, como en cualquier herramienta de bajo nivel, existen dos modalidades operativas diametralmente distintas:

1. **Expansión o contracción sobre el extremo final:** Cuando se amplía o reduce una partición modificando únicamente su sector de finalización (*Last LBA*), el sector inicial (*First LBA*) permanece intacto. En este caso, la operación es casi instantánea: el software simplemente ajusta los metadatos del sistema de archivos mediante `resize2fs` o `ntfsresize` para declarar un nuevo conteo de bloques y actualiza el descriptor en la tabla GPT.
2. **Desplazamiento del sector inicial (reubicación de partición):** Cuando una partición es movida espacialmente o reducida desde su extremo izquierdo, su sector inicial cambia físicamente de posición. Esta operación exige que GParted lea y copie físicamente cada bloque de datos y metadato desde su ubicación original hacia las nuevas coordenadas de sector. En volúmenes de cientos de gigabytes, este proceso puede demorar varias horas de I/O intensiva. Si durante esta copia masiva ocurre una interrupción de energía eléctrica, un reinicio inesperado o un fallo de hardware, la par-

tición quedará bifurcada entre sectores antiguos y nuevos con estructuras truncadas, destruyendo el sistema de archivos de forma prácticamente irrecuperable.

Por esta razón técnica, en un esquema de arranque dual nunca debe intentarse desplazar el sector inicial de las particiones reservadas o de sistema de Windows. La liberación de espacio debe limitarse estrictamente a contraer el extremo final del volumen C:, generando espacio contiguo libre hacia la derecha sin alterar el punto de montaje de inicio ni los metadatos de arranque preexistentes.

5. Protocolo técnico de instalación y configuración paso a paso

La instalación manual de Arch Linux en una máquina física con Windows preinstalado constituye un ejercicio de ingeniería de sistemas riguroso. Al carecer de un asistente automatizado con interfaz gráfica, cada fase del despliegue requiere la ejecución consciente de comandos de bajo nivel que enlazan el hardware anfitrión, las estructuras de particionamiento, el subsistema de memoria virtual y la configuración de arranque unificado (Arch Linux Project, 2026b). En las siguientes subsecciones se detalla el protocolo técnico paso a paso para consumar la instalación de forma reproducible y segura.

5.1. Generación del medio de arranque y verificación del modo de inicio UEFI

El primer paso consiste en transferir la imagen ISO oficial de Arch Linux hacia un dispositivo de almacenamiento USB para convertirlo en un medio de arranque híbrido. Las imágenes de Arch Linux adoptan la especificación ISOHybrid, lo que significa que el archivo ISO contiene tanto una tabla de particiones MBR/GPT en sus primeros sectores como estructuras de sistema de archivos ISO 9660 y particiones de arranque EFI (FAT).

En sistemas tipo UNIX/Linux, la transferencia debe ejecutarse a nivel de bloques mediante la utilidad `dd`, escribiendo directamente sobre el nodo raíz del dispositivo USB (p. ej., `/dev/sdb`) y nunca sobre una partición individual (como `/dev/sdb1`). En sistemas Windows, se recomienda emplear la utilidad de código abierto Rufus configurada estrictamente en modo “Imagen DD”; seleccionar el modo estándar de extracción ISO corrompe los cargadores híbridos de Arch Linux e impide el arranque correcto en placas base con firmware UEFI estricto:

```
# Escribir la imagen ISO en el dispositivo USB a nivel de bloques crudos
dd if=archlinux-2026.09.01-x86_64.iso of=/dev/sdb bs=4M status=progress oflag=sync
```

Con el medio USB insertado en el equipo, se presiona la tecla de interrupción de arranque de la placa base durante el encendido (habitualmente F12, F11, Esc o F2) para acceder al menú de selección de dispositivos de inicio UEFI (*UEFI Boot Menu*). En este menú es mandatorio seleccionar la entrada que incluya el prefijo `UEFI:` (p. ej., `UEFI: Generic Flash Disk, Partition 1`).

Tras completar la inicialización del núcleo en memoria y acceder a la consola interactiva (`root@archiso`), se ejecutan los comandos preliminares de preparación del entorno de trabajo:

```
# 1. Establecer la distribución de teclado en español para la consola
loadkeys la-latin1

# 2. Verificar de manera inequívoca que el sistema inició bajo UEFI nativo (64 bits)
cat /sys/firmware/efi/fw_platform_size
# Salida esperada: 64

# 3. Configurar la interfaz inalámbrica si no se dispone de conexión por cable
iwctl -passphrase çlave_red"station wlan0 connect "Nombre_SSID"

# 4. Probar la conectividad a Internet mediante el protocolo ICMP
ping -c 3 archlinux.org

# 5. Sincronizar el reloj del sistema mediante el protocolo NTP
timedatectl set-ntp true
```

La confirmación del valor `64` en el archivo `fw_platform_size` certifica que el núcleo Linux opera con acceso irrestricto a los servicios de tiempo de ejecución de UEFI en modo largo de 64 bits, habilitando la futura instalación del cargador de arranque en la memoria NVRAM.

5.2. Particionamiento manual y formateo de sistemas de archivos

Una vez verificado el espacio no asignado liberado previamente desde Windows (Sección 4.1), se procede al particionamiento manual del disco de estado sólido (en este protocolo, representado por la unidad NVMe `/dev/nvme0n1`).

Mediante la herramienta especializada `gdisk`, se crean las tres particiones requeridas para el entorno Linux aprovechando el espacio remanente continuo:

1. **Partición de intercambio (`/dev/nvme0n1p4`):** Asignación de 8 GiB con código de tipo `8200` (*Linux swap*).
2. **Partición raíz (`/dev/nvme0n1p5`):** Asignación de 60 GiB con código de tipo `8304` (*Linux x86-64 root /*).
3. **Partición de usuario (`/dev/nvme0n1p6`):** Asignación del remanente del disco con código de tipo `8302` (*Linux /home*).

A continuación, se inspeccionan los descriptores de bloque y se procede al formateo de bajo nivel. Es una directiva de seguridad crítica recordar que la partición del sistema EFI compartida (`/dev/nvme0n1p1`) **jamás debe ser formateada**, ya que alberga los ejecutables de Windows Boot Manager; únicamente las particiones nuevas de Linux deben ser inicializadas:

```
# 1. Inspeccionar el mapa de particiones antes del formateo
lsblk -f /dev/nvme0n1

# 2. Inicializar la partición de paginación e intercambio (swap)
mkswap -L LINUX_SWAP /dev/nvme0n1p4
swapon /dev/nvme0n1p4

# 3. Formatear la partición raíz en ext4 con bloques de 4096 bytes y registro en diario
mkfs.ext4 -L LINUX_ROOT /dev/nvme0n1p5

# 4. Formatear la partición de datos de usuario en ext4
mkfs.ext4 -L LINUX_HOME /dev/nvme0n1p6
```

El formateo mediante `mkfs.ext4` genera de forma determinista el superbloque maestro,

las tablas de inodos y el archivo de diario (*journal*), asignando etiquetas legibles (`LINUX_ROOT` y `LINUX_HOME`) y generando identificadores universales únicos (UUID) que garantizarán la estabilidad del montaje posterior.

5.3. Jerarquía de montaje en /mnt y despliegue base (pacstrap)

Para instalar los paquetes del sistema operativo en el disco de destino, Linux implementa el concepto de árbol de montaje provisional, anclado de forma estandarizada en el directorio `/mnt` del sistema en vivo (Silberschatz et al., 2006, §12.1, p. 409). El orden de montaje debe obedecer estrictamente la jerarquía del árbol de directorios del Estándar de Jerarquía del Sistema de Archivos (FHS): la raíz del sistema de archivos debe montarse primero, y los puntos de montaje subsidiarios deben crearse dentro de ella antes de recibir sus respectivos sistemas de archivos:

```
# 1. Montar la partición raíz de Linux en el punto base /mnt
mount /dev/nvme0n1p5 /mnt

# 2. Crear los directorios secundarios para el cargador de arranque y usuarios
mkdir -p /mnt/boot
mkdir -p /mnt/home

# 3. Montar la partición ESP compartida de Windows en /mnt/boot
mount /dev/nvme0n1p1 /mnt/boot

# 4. Montar la partición de usuario en /mnt/home
mount /dev/nvme0n1p6 /mnt/home
```

Con los volúmenes montados coherentemente bajo `/mnt`, se procede a optimizar la lista de servidores réplica con `reflector` para maximizar la velocidad de transferencia y se invoca la utilidad central `pacstrap` para descargar y desplegar dentro de `/mnt` la colección fundamental de software requerida para dotar a la máquina de autonomía operativa:

```
# 1. Filtrar y ordenar los 10 espejos HTTPS más veloces sincronizados recientemente
reflector -country Mexico,US -protocol https -latest 10 -sort rate ^
   -save /etc/pacman.d/mirrorlist

# 2. Desplegar el sistema operativo base y herramientas esenciales en el disco
pacstrap -K /mnt base linux linux-firmware base-devel nano networkmanager ^
   grub efibootmgr dosfstools os-prober
```

La selección de paquetes responde a funciones técnicas específicas:

- **Sistema base (`base`):** Meta-paquete que instala el árbol jerárquico FHS, librerías GNU C (`glibc`) y utilidades esenciales del sistema (`coreutils`, `systemd`, `iproute2`).
- **Núcleo y microcódigo (`linux`, `linux-firmware`):** Binario ejecutable del núcleo Linux monolítico modular (`vmlinuz-linux`) y controladores propietarios en formato binario (*blobs*) indispensables para interfaces Wi-Fi, audio y aceleración de gráficos.
- **Herramientas de construcción (`base-devel`):** Herramientas de compilación (`gcc`, `make`, `binutils`, `fakeroot`) necesarias para la posterior construcción de módulos del núcleo y paquetes del repositorio de usuarios de Arch (AUR).
- **Gestor de red (`networkmanager`):** Demonio del sistema operativo para la gestión integral de redes cableadas e inalámbricas.

- **Pila de arranque (**`grub`**,** `efibootmgr`**,** `dosfstools`**,** `os-prober`**):** Colección de bajo nivel destinada a gobernar el arranque dual en plataformas UEFI.

Para concluir esta fase, se genera el archivo de definición estática de sistemas de archivos (`/etc/fstab`) empleando la herramienta `genfstab`:

```
# Generar la tabla fstab empleando identificadores universales únicos (UUID)
genfstab -U /mnt » /mnt/etc/fstab

# Inspeccionar el contenido generado para verificar la corrección de los montajes
cat /mnt/etc/fstab
```

El modificador `-U` resulta estrictamente mandatorio: instruye al generador a identificar cada volumen mediante su UUID de 128 bits en lugar de nombres de dispositivo dinámicos (como `/dev/sda` o `/dev/nvme0n1`). Debido a que los índices de hardware pueden reordenarse aleatoriamente durante el encendido si se conectan dispositivos periféricos o varían las secuencias de inicialización del bus PCIe, el empleo de UUID garantiza que el sistema operativo monte invariablemente el volumen correcto en su respectivo punto de anclaje, previniendo fallos catastróficos de montaje durante el arranque (Stallings, 2018, §11.5, pp. 521–523).

5.4. Configuración del entorno en chroot

Para configurar las variables del nuevo sistema operativo (zonas horarias, cuentas de usuario, servicios de inicio e imágenes de memoria inicial), es necesario cambiar la raíz operativa del proceso hacia el sistema instalado. Esto se ejecuta mediante el comando `arch-chroot`:

```
# Ingresar en el entorno enjaulado del sistema instalado en disco
arch-chroot /mnt
```

El mecanismo `chroot` (*change root*) redefine el directorio raíz `/` para el intérprete interactivo y sus subprocesos, montando automáticamente los pseudo-sistemas de archivos del núcleo (`/proc`, `/sys`, `/dev` y `/run`) dentro de la jaula. Una vez dentro de `arch-chroot`, se ejecutan secuencialmente los siguientes pasos de configuración:

5.4.1. Sincronización horaria y localización geográfica

Se configura la zona horaria enlazando el descriptor correspondiente de la base de datos de zonas horarias (`/usr/share/zoneinfo/`), se sincroniza el reloj de hardware en tiempo universal coordinado (UTC), se descomentan las líneas `es_MX.UTF-8 UTF-8` y `en_US.UTF-8 UTF-8` en `/etc/locale.gen`, y se declaran las variables persistentes de localización y distribución de teclado en consola:

```
# 1. Enlazar la zona horaria de Ciudad de México y sincronizar el RTC en UTC
ln -sf /usr/share/zoneinfo/America/Mexico_City /etc/localtime
hwclock -systohc -utc

# 2. Generar los descriptores de localización y establecer variables de entorno
locale-gen
echo "LANG=es_MX.UTF-8» /etc/locale.conf
echo "KEYMAP=la-latin1» /etc/vconsole.conf
```

5.4.2. Identidad de red, seguridad y servicios del sistema

Se define el nombre de nodo de la máquina en la red local (*hostname*) y se habilita el demonio de gestión de redes `NetworkManager`. Por estrictas razones de seguridad operacional, operar cotidianamente con el superusuario privilegiado `root` es una práctica desaconsejada; por ello, se fija una contraseña para `root` y se crea una cuenta de trabajo ordinaria incorporada al grupo suplementario `wheel`, habilitando la delegación controlada de privilegios administrativos mediante `sudo`:

```
# 1. Definir el nombre del equipo y habilitar el servicio de red en systemd
echo .estacion-arch» /etc/hostname
systemctl enable NetworkManager.service

# 2. Asignar contraseña a root y crear usuario ordinario en el grupo wheel
passwd
useradd -m -g users -G wheel -s /bin/bash oscar
passwd oscar

# 3. Autorizar la ejecución de privilegios administrativos mediante visudo
echo "%wheel ALL=(ALL:ALL) ALL» /etc/sudoers.d/10-wheel-privileges
```

5.4.3. Generación de la imagen de disco en memoria inicial (initramfs)

El archivo `/etc/mkinitcpio.conf` define los módulos del núcleo y los ganchos (*hooks*) que se empaquetarán en la imagen de memoria inicial (`initramfs-linux.img`). Durante el encendido temprano, el núcleo Linux monta esta imagen temporal en la memoria RAM para cargar los controladores del bus NVMe, controladores de sistemas de archivos ext4 y protocolos de teclado antes de transferir el control al sistema de archivos raíz definitivo (Deitel et al., 2004, §2.4.3, pp. 76–77). Se compilan las imágenes del núcleo ejecutando:

```
# Generar las imágenes de memoria inicial (initramfs) para todos los núcleos
mkinitcpio -P
```

5.5. Instalación de GNU GRUB con soporte para Windows Boot Manager (os-prober)

La fase culminante de la instalación consiste en desplegar el gestor de arranque GNU GRUB 2 en la partición ESP compartida y articular la detección automática del cargador de Windows (Free Software Foundation, 2024a).

En primer lugar, se activa el soporte de sondeo de sistemas alternos en el archivo de configuración declarativo de GRUB, eliminando la restricción de seguridad predeterminada de `os-prober`. Luego, se procede a la instalación física del binario UEFI en la partición ESP compartida (montada en `/boot`) y se genera el archivo de menú ejecutable:

```
# 1. Habilitar la directiva para que os-prober explore particiones foráneas
echo "GRUB_DISABLE_OS_PROBER=false»> /etc/default/grub

# 2. Instalar el binario de GRUB en la partición EFI y registrar la entrada NVRAM
grub-install -target=x86_64-efi -efi-directory=/boot -bootloader-id=GRUB -recheck
```

```
# 3. Compilar la configuración de GRUB integrando Linux y Windows Boot Manager
grub-mkconfig -o /boot/grub/grub.cfg
```

Durante la ejecución de este comando, la consola debe reportar de manera explícita la detección del núcleo Linux y la detección del cargador de Windows:

```
Found linux image: /boot/vmlinuz-linux
Found initrd image: /boot/initramfs-linux.img
Found Windows Boot Manager on /boot/EFI/Microsoft/Boot/bootmgfw.efi@/dev/nvme0n1p1
done
```

Esta salida confirma que el script modular `30_os-prober` generó exitosamente la entrada de carga encadenada (*chainloader*), asegurando que al encender el equipo el usuario podrá conmutar indistintamente entre Arch Linux y Microsoft Windows desde un único menú unificado.

Concluida la configuración, se finaliza la sesión enjaulada, se desmontan de forma limpia todos los sistemas de archivos para vaciar los búferes de memoria hacia el almacenamiento no volátil y se reinicia la máquina:

```
# Salir del entorno chroot
exit

# Desmontar recursivamente la jerarquía de directorios
umount -R /mnt

# Reiniciar el equipo para ingresar al nuevo sistema operativo
reboot
```

6. Postinstalación, configuración del sistema y resolución de fallos

Tras concluir la secuencia de instalación y reiniciar el equipo, el sistema operativo ingresa en su fase de producción. En este punto, Arch Linux se presenta como una instalación base minimalista en modo consola. A diferencia de las distribuciones convencionales de propósito general que imponen un entorno gráfico homogéneo y servicios predeterminados, la arquitectura de Arch Linux delega enteramente en el usuario la responsabilidad de seleccionar y modularizar los componentes del espacio de usuario, optimizando el rendimiento de la máquina y garantizando una coexistencia armónica con Windows (Arch Linux Project, 2026b).

6.1. Gestión de paquetes con Pacman y modularidad en la configuración del entorno

La gestión del ciclo de vida del software en Arch Linux se articula a través de Pacman, un gestor de paquetes de alto rendimiento escrito en lenguaje C que combina un formato de compresión binario simple con un motor dinámico de resolución de dependencias (Arch Linux Project, 2026b). A través de la invocación de flags mnemotécnicos en mayúsculas y minúsculas, Pacman administra la sincronización con los repositorios remotos oficiales (`core`, `extra`, `multilib`):

```
# Sincronizar bases de datos y actualizar todos los paquetes del sistema
sudo pacman -Syu

# Buscar un paquete en los repositorios remotos oficiales
pacman -Ss nombre_paquete

# Instalar un paquete resolviendo dependencias de forma automática
sudo pacman -S nombre_paquete

# Desinstalar un paquete eliminando sus dependencias huérfanas y configuraciones
sudo pacman -Rns nombre_paquete
```

La principal fortaleza arquitectónica de Arch Linux radica en su versatilidad: no existe una configuración gráfica canónica u obligatoria. El ingeniero o administrador puede concebir su estación de trabajo eligiendo libremente entre múltiples paradigmas de interacción:

- **Entornos de escritorio integrados (DE):** Soluciones completas que incorporan gestores de ventanas, barras de tareas, centros de control y pilas de aplicaciones coherentes. Entre ellos destacan **KDE Plasma** (basado en el framework Qt, altamente modular, eficiente en el uso de memoria RAM y con soporte nativo para Wayland), **GNOME** (construido sobre GTK, con una experiencia de usuario orientada a flujos de trabajo limpios) y **XFCE** (orientado a máxima ligereza y estabilidad tradicional).
- **Compositores modernos e independientes de Wayland:** Para flujos de trabajo de ingeniería de software donde se prioriza la navegación por teclado y el aprovechamiento óptimo de los monitores de alta resolución, es posible prescindir de entornos de escritorio pesados y desplegar directamente compositores de ventanas en mosaico (*tiling window managers*), tales como **Hyprland** (compositor dinámico con aceleración por hardware) o **Sway** (reemplazo moderno y conforme a Wayland del clásico `i3-wm`).
- **Gestores de ventanas tradicionales sobre X11:** Alternativas altamente depuradas como **i3**, **bspwm** o **dwm**, que consumen menos de 100 MiB de memoria RAM en reposo.
- **Estación de trabajo sin entorno gráfico (Headless / CLI):** En servidores de cálculo o entornos de investigación, la máquina puede mantenerse puramente en modo consola, explotando multiplexores de terminal como `tmux` o `zellij`, acceso remoto cifrado mediante `OpenSSH` y motores de contenedores (`podman` o `docker`), prescindiendo por completo de la sobrecarga del servidor gráfico.

Asimismo, el ecosistema de Arch Linux se complementa con el Repositorio de Usuarios de Arch (AUR, *Arch User Repository*), una plataforma comunitaria que alberga más de 80 000 recetas declarativas de compilación (`PKGBUILD`). Mediante utilidades auxiliares como `paru` o `yay`, los usuarios pueden compilar, validar firmas criptográficas e instalar controladores especializados, herramientas de desarrollo y software de terceros con la misma comodidad que los paquetes oficiales.

6.2. Ajuste del cargador de arranque y sincronización de reloj de hardware

Para perfeccionar la convivencia diaria en arranque dual, el archivo de configuración declarativo de GNU GRUB (`/etc/default/grub`) admite parámetros avanzados que elevan la ergonomía operativa (Free Software Foundation, 2024a):

1. **Memoria del último sistema iniciado:** En lugar de fijar una opción predeterminada

estática, se configuran las variables `GRUB_DEFAULT=saved` y `GRUB_SAVEDEFAULT=true`. Esta directiva instruye a GRUB a registrar en disco el último sistema booteado, evitando intervenciones manuales durante reinicios sucesivos.

2. **Temporizador de espera:** Se modula el parámetro `GRUB_TIMEOUT=5` para fijar un intervalo de 5 segundos antes de disparar el arranque automático de la opción seleccionada.

```
# Configurar GRUB para recordar automáticamente el último sistema operativo booteado
sudo sed -i 's/^GRUB_DEFAULT=.*/GRUB_DEFAULT=saved/' /etc/default/grub
sudo sed -i 's/^GRUB_TIMEOUT=.*/GRUB_TIMEOUT=5/' /etc/default/grub
echo "GRUB_SAVEDEFAULT=true"| sudo tee -a /etc/default/grub

# Regenerar el archivo grub.cfg en la partición ESP compartida
sudo grub-mkconfig -o /boot/grub/grub.cfg
```

En paralelo, es imperativo certificar que la sincronización horaria opere de acuerdo con el estándar UTC. Mediante la utilidad `timedatectl status`, se verifica que el reloj de tiempo real (RTC) no se encuentre operando en hora local (`RTC in local TZ: no`) y que el servicio de sincronización por red mediante protocolo NTP se encuentre activo (`NTP service: active`), previniendo las derivas horarias entre Windows y Linux examinadas en la Sección 2.3.

6.3. Diagnóstico y resolución de incidencias habituales en arranque dual

En el ciclo de vida de una estación de trabajo con arranque múltiple, las actualizaciones del sistema operativo comercial o alteraciones en el firmware pueden desencadenar fallos de arranque conocidos. A continuación se documenta el tratamiento técnico de las dos incidencias más recurrentes:

6.3.1. Sobrescritura del orden de arranque por actualizaciones de Windows

Es habitual que las actualizaciones mayores acumulativas de Windows reescriban de forma unilateral las variables del firmware UEFI en la memoria NVRAM, asignando la prioridad máxima a `Windows Boot Manager` (`Boot0000`) y marginando al cargador GRUB. En consecuencia, la máquina reinicia directamente en Windows sin mostrar el menú de arranque unificado.

Para restituir el orden correcto sin necesidad de reinstalar el sistema, se inicia el equipo desde el medio USB de Arch Linux o se accede a una consola administrativa y se invoca `efibootmgr` (Unified Extensible Firmware Interface Forum, 2024):

```
# 1. Listar las entradas UEFI registradas en la NVRAM y el orden actual
sudo efibootmgr -v

# 2. Suponiendo que GRUB tiene el ID 0001 y Windows el ID 0000, restablecer prioridad
sudo efibootmgr -o 0001,0000

# 3. Recrear el descriptor si fue eliminado de la NVRAM (ESP en /dev/nvme0n1p1)
sudo efibootmgr -c -d /dev/nvme0n1 -p 1 -L "GRUB1 "\EFI\GRUB\grubx64.efi"
```

6.3.2. Reactivación accidental de Fast Startup y volumen NTFS sucio

Si un usuario o actualización de Windows reactiva de forma desapercibida la función de inicio rápido (Fast Startup), los sistemas de archivos NTFS quedarán congelados en estado hibernado. Al arrancar Arch Linux e intentar montar una partición de datos de Windows, el núcleo emitirá un fallo de volumen sucio (`cannot mount read-write, volume is dirty / hibernated`).

En esta contingencia, **jamás debe forzarse el montaje en modo lectura/escritura mediante utilidades de reparación como** `ntfsfix`, puesto que truncar el diario de transacciones provocará la corrupción irreversible del estado que Windows mantiene congelado en memoria RAM. El protocolo de remediación exige:

1. Montar provisionalmente la partición en modo estrictamente de solo lectura (`mount -o ro /dev/nvme0n1p3 /mnt/datos`) si se requiere consultar urgentemente un archivo.
2. Reiniciar el equipo en Windows, abrir una consola de comandos con privilegios elevados y desactivar definitivamente la hibernación invocando `powercfg /h off`.
3. Apagar la máquina por completo mediante la instrucción de consola `shutdown /s /f /t 0`, garantizando que todos los volúmenes NTFS se desmonten limpiamente en el disco físico antes de retornar a Linux.

7. Conclusiones personales

El desarrollo del presente manual técnico y el despliegue práctico de una estación con arranque dual entre Microsoft Windows y Arch Linux consolidan una comprensión integral y de bajo nivel sobre la interacción entre el firmware, el subsistema de almacenamiento masivo y los núcleos de los sistemas operativos contemporáneos. La instalación manual desasistida desmitifica las abstracciones que los instaladores automáticos ocultan, demostrando que el medio secundario no es un receptáculo homogéneo sino una jerarquía rigurosa: la transición de CHS a LBA48 y la alineación a 2048 sectores (1 MiB) resultan determinantes para optimizar la capa de traducción de memoria flash (FTL) en unidades de estado sólido. Asimismo, el estándar GPT demuestra su superioridad frente a MBR al superar la barrera de los 2 TiB, asignar GUID universales de 128 bits e integrar redundancia estructural con sumas CRC32 que erradican la corrupción catastrófica de particiones.

En el ámbito del encendido, la adopción del estándar UEFI de 64 bits con controladores FAT nativos en la partición ESP compartida transforma cualitativamente la arquitectura de arranque múltiple. GNU GRUB prescinde de sectores ciegos y se articula como un ejecutable estandarizado que convive con Windows Boot Manager mediante carga encadenada (chainloading) y gestión determinista en la NVRAM con `efibootmgr`. Esta coexistencia evidencia la necesidad de anticipar riesgos en sistemas heterogéneos: la desactivación obligatoria del inicio rápido de Windows (Fast Startup) para evitar corrupciones en volúmenes NTFS congelados, la sincronización del reloj de hardware (RTC) en UTC y el respaldo atómico del almacén BCD y tablas GPT con `sgdisk`.

Finalmente, la ejecución metódica de cada fase —desde la verificación criptográfica con SHA-256 y firmas OpenPGP hasta el despliegue del sistema base con `pacstrap`, la configuración en `chroot` y la generación de imágenes con `mkinitcpio`— ratifica el valor formativo de Arch Linux. Al delegar las decisiones arquitectónicas en el administrador sin esquemas prefabricados, se consolida una autonomía insustituible para el diagnóstico, la administración y el diseño de infraestructuras en el ejercicio profesional.

Referencias

Arch Linux Project. (2026a). *Arch linux downloads and release verification*. Consultado el 1 de noviembre de 2026, desde https://archlinux.org/download/

Arch Linux Project. (2026b). *Installation guide*. Consultado el 1 de noviembre de 2026, desde https://wiki.archlinux.org/title/Installation_guide

Carretero Pérez, J., García Carballeira, F., Miguel Anasagasti, P. d., & Pérez Costoya, F. (2001). *Sistemas operativos: una visión aplicada* (1.ª ed.). McGraw-Hill/Interamericana de España.

Deitel, H. M., Deitel, P. J., & Choffnes, D. R. (2004). *Operating systems* (3.ª ed.). Pearson/Prentice Hall.

Free Software Foundation. (2024a). *Gnu grub manual 2.12*. Consultado el 1 de noviembre de 2026, desde https://www.gnu.org/software/grub/manual/grub/grub.html

Free Software Foundation. (2024b). *Gnu parted user's manual*. Consultado el 1 de noviembre de 2026, desde https://www.gnu.org/software/parted/manual/parted.html

GParted Project. (2024). *Gparted manual*. Consultado el 1 de noviembre de 2026, desde https://gparted.org/display-doc.php?name=help-manual

Linux Foundation. (2015). *Filesystem Hierarchy Standard (FHS)* (inf. téc. N.º Version 3.0). Linux Foundation. Consultado el 1 de noviembre de 2026, desde https://refspecs.linuxfoundation.org/FHS_3.0/fhs-3.0.pdf

McHoes, A. M., & Flynn, I. M. (2011). *Sistemas operativos* (6.ª ed.). Cengage Learning Editores.

Microsoft Corporation. (2024a). *Bcdedit command-line options*. Consultado el 1 de noviembre de 2026, desde https://learn.microsoft.com/en-us/windows-hardware/manufacture/desktop/bcdedit-command-line-options-techref-di

Microsoft Corporation. (2024b). *Diskpart command-line options*. Consultado el 1 de noviembre de 2026, desde https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/diskpart

Microsoft Corporation. (2024c). *Overview of boot options in windows*. Consultado el 1 de noviembre de 2026, desde https://learn.microsoft.com/en-us/windows-hardware/drivers/bringup/boot-options-in-a-multiboot-environment

Silberschatz, A., Galvin, P. B., & Gagne, G. (2006). *Fundamentos de sistemas operativos* (7.ª ed.). McGraw-Hill/Interamericana de España.

Smith, R. W. (2022). *Gpt fdisk tutorial*. Consultado el 1 de noviembre de 2026, desde https://www.rodsbooks.com/gdisk/

Stallings, W. (2018). *Operating systems: Internals and design principles* (9.ª ed.) [Global Edition]. Pearson Education Limited.

Unified Extensible Firmware Interface Forum. (2024). *Unified Extensible Firmware Interface (UEFI) Specification* (inf. téc. N.º Version 2.10). UEFI Forum. Consultado el 1 de noviembre de 2026, desde https://uefi.org/specifications